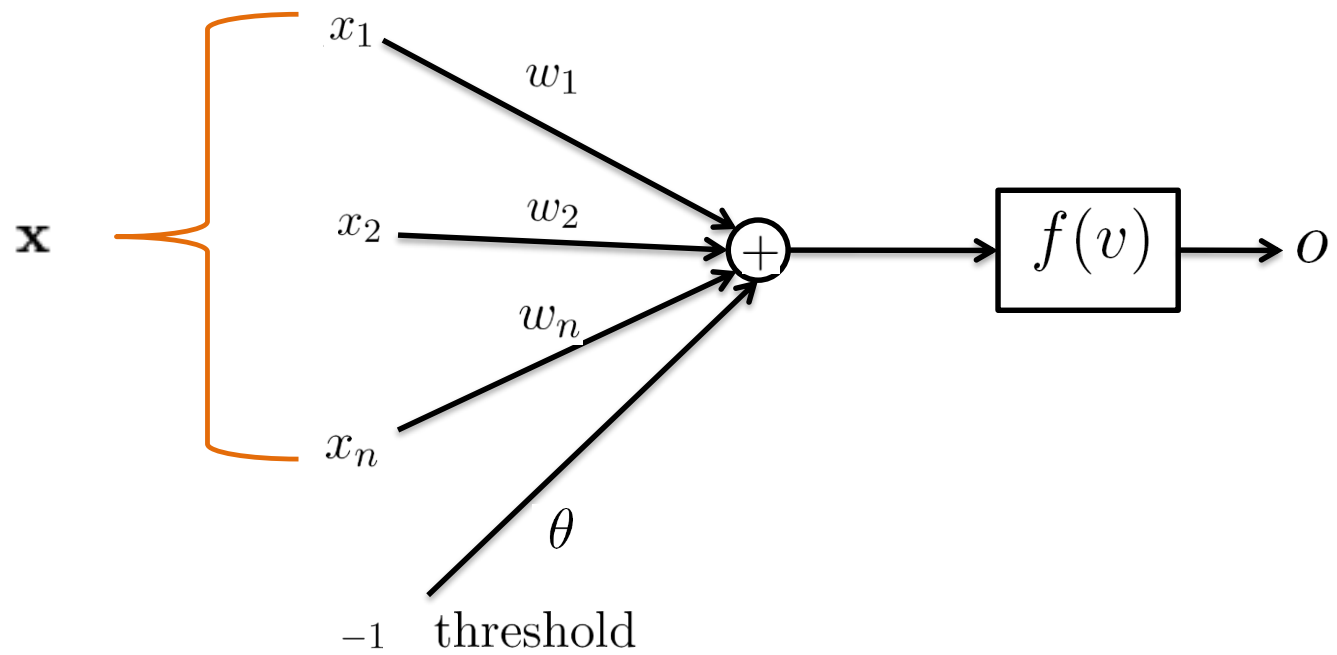
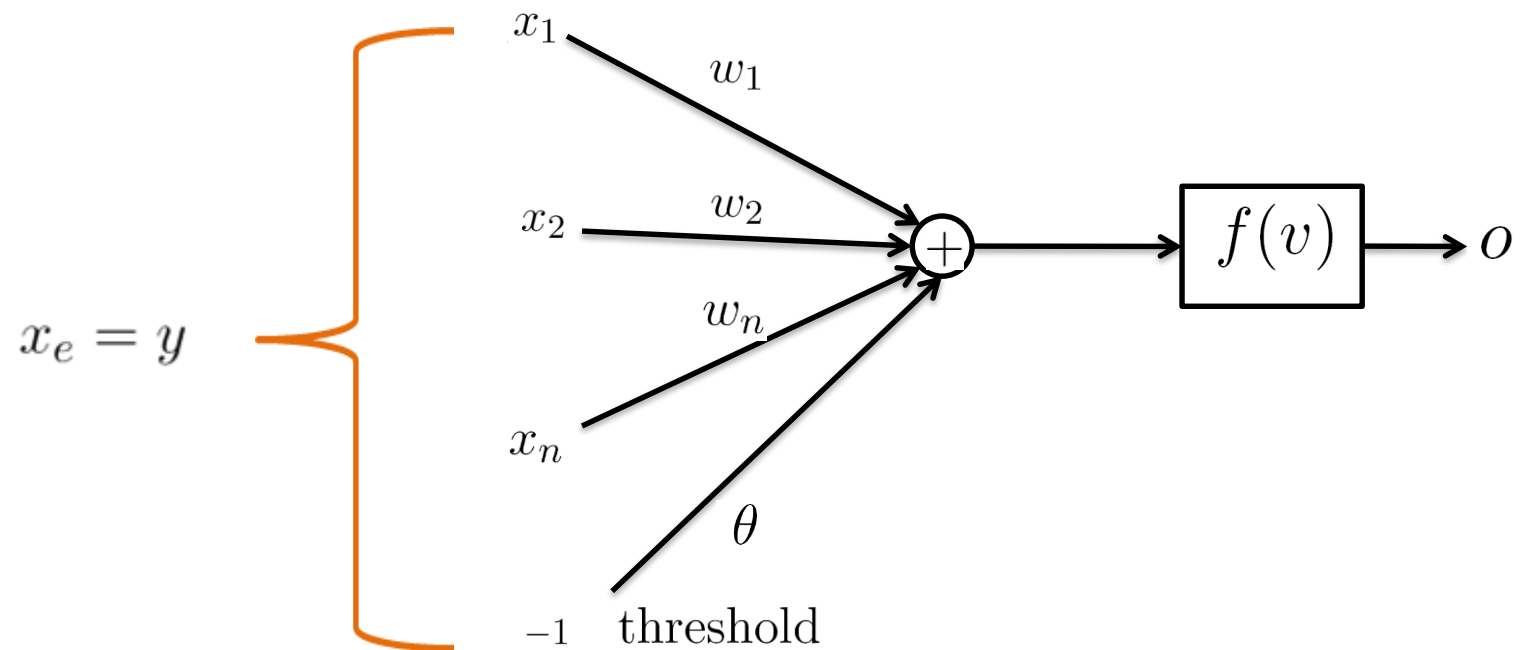
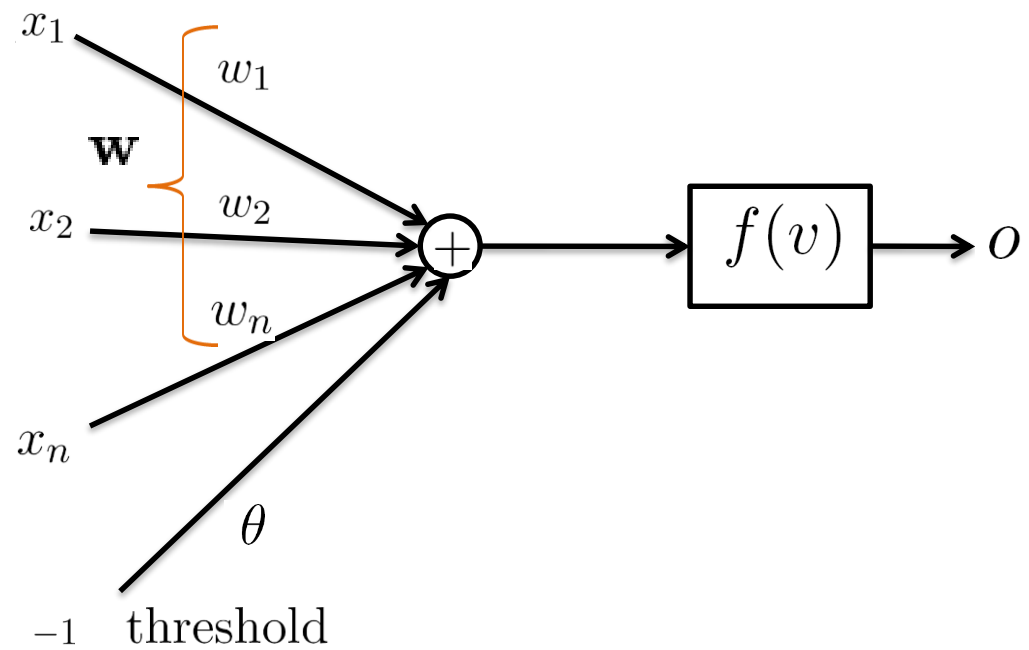


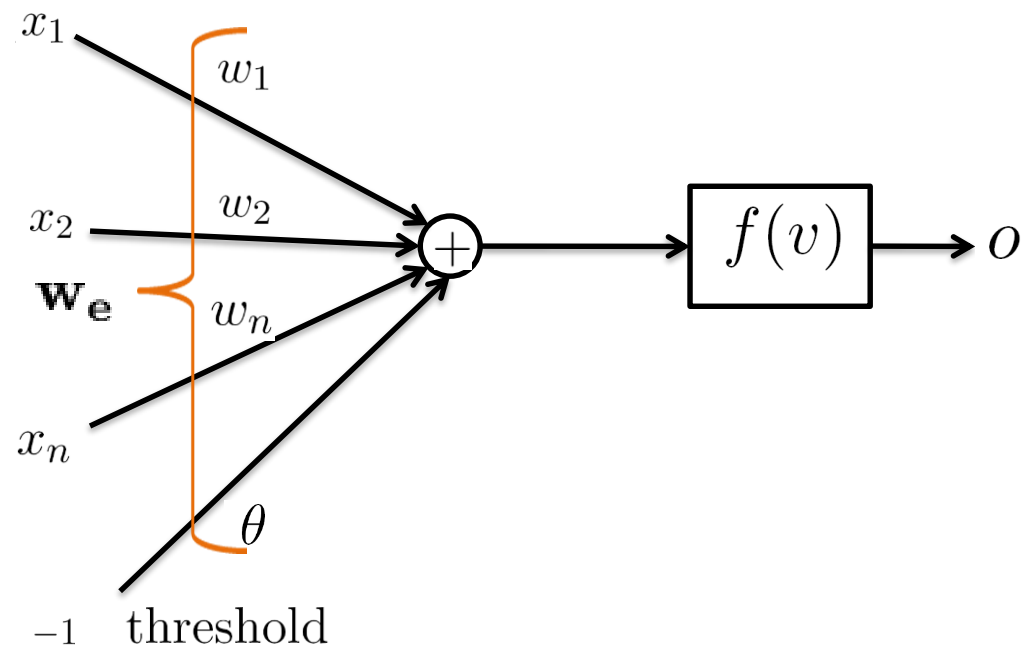
➤ PERCEPTRON CLASSIFIERS

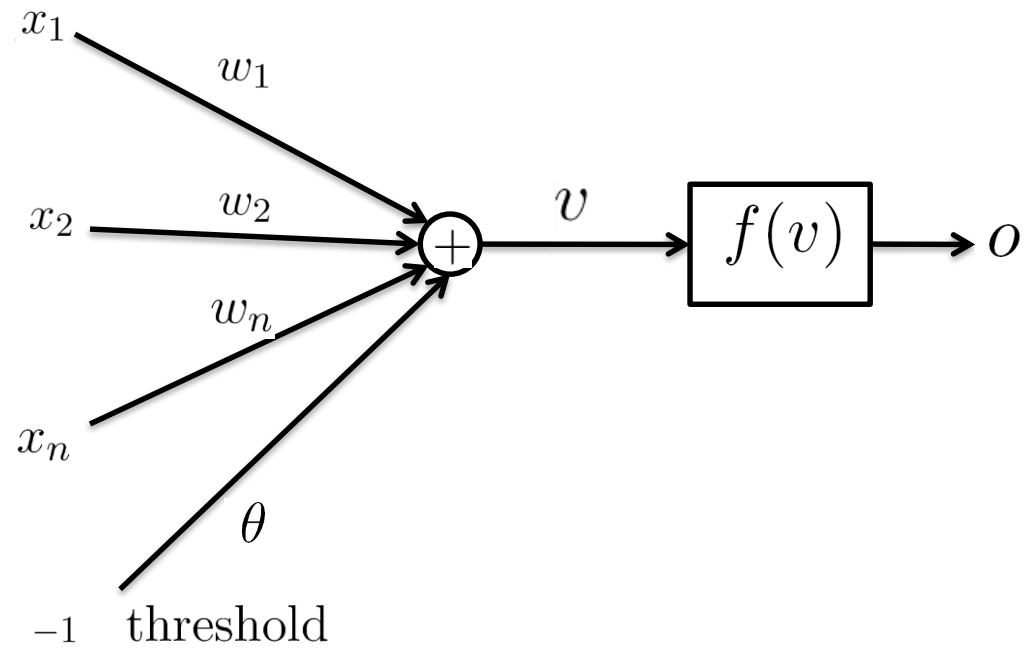
- Perceptron
- Single Neuron Classification
- Single Neuron Perceptron Learning Algorithm
- Convergence
- Single Layer/Multicategory Case







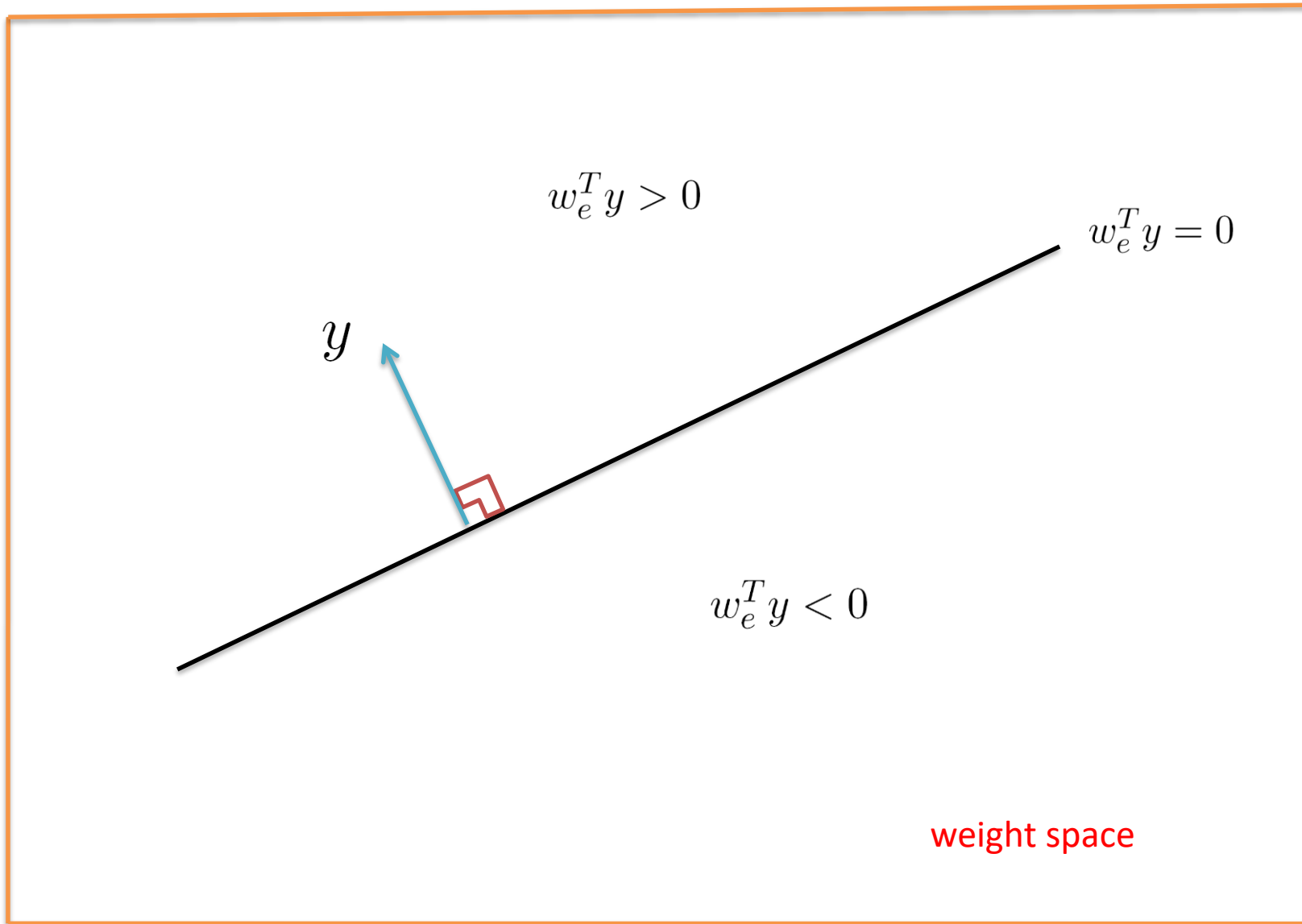


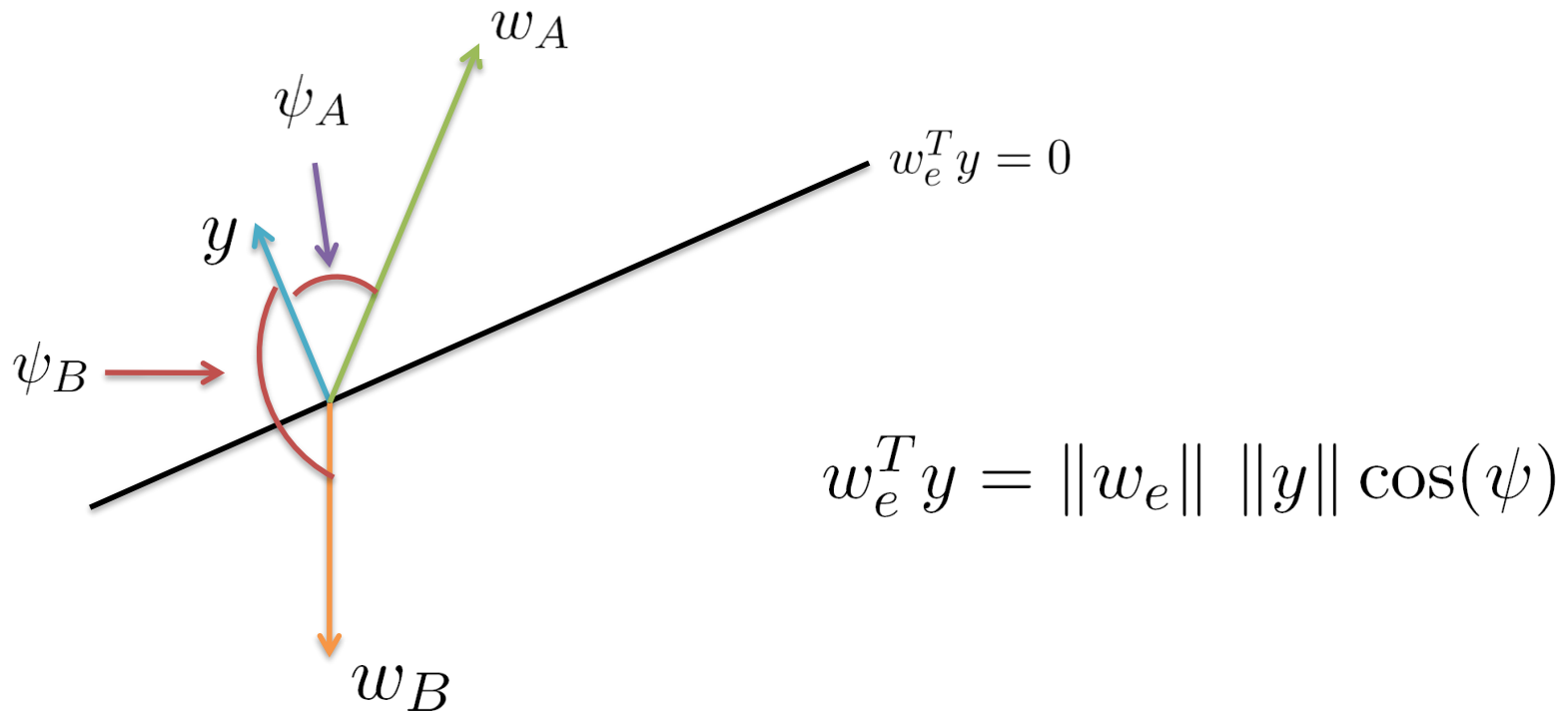


$$v = w_1x_1 + w_2x_2 + \dots + w_nx_n - \theta$$
$$o = f(v) = \text{sgn}(v)$$

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad w = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix} \quad \Rightarrow \quad v = w^T x - \theta$$

$$x_e = y = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ -1 \end{pmatrix} \quad w_e = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ \theta \end{pmatrix} \quad \Rightarrow \quad \begin{aligned} v &= w^T x - \theta \\ &= w_e^T x_e = w_e^T y \end{aligned}$$





$$|\psi_A| < 90^\circ \quad \Leftrightarrow \quad w_A^T y > 0$$

$$|\psi_B| > 90^\circ \quad \Leftrightarrow \quad w_B^T y < 0$$

- Perceptron Classification Problem :

- Given two classes $\left\{ \begin{array}{l} \mathcal{C}_1 = \{x^1, x^2, \dots, x^m\} \\ \mathcal{C}_2 = \{x^{m+1}, x^{m+2}, \dots, x^N\} \end{array} \right.$

- Extended inputs $\left\{ \begin{array}{l} \mathcal{C}_1 = \{y^1, y^2, \dots, y^m\} \\ \mathcal{C}_2 = \{y^{m+1}, y^{m+2}, \dots, y^N\} \end{array} \right.$

- Perceptron Classification Problem :

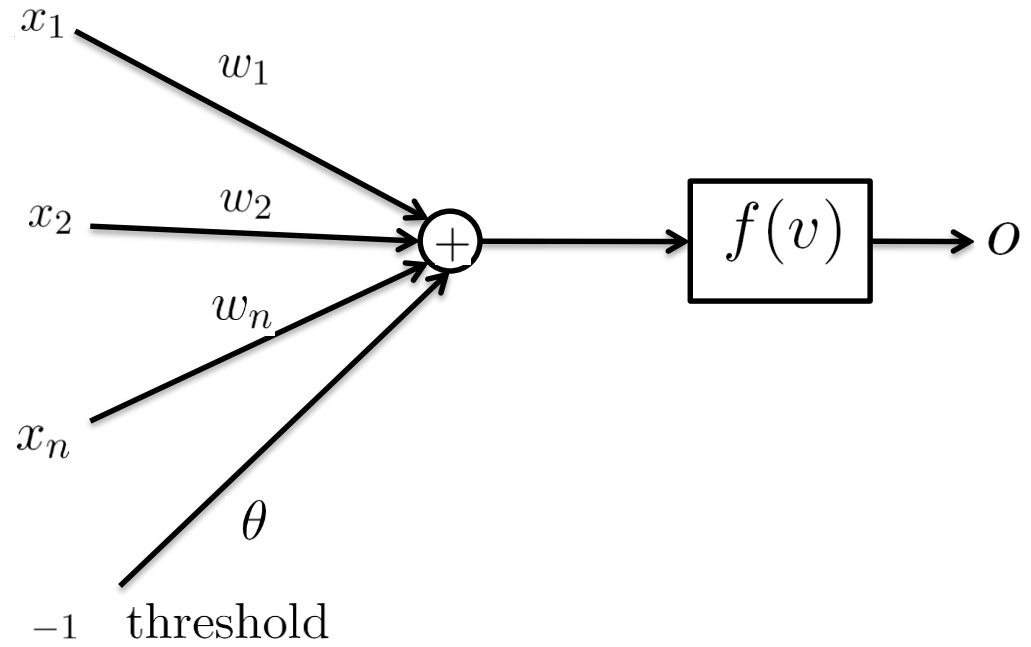
- Given two classes $\left\{ \begin{array}{l} \mathcal{C}_1 = \{x^1, x^2, \dots, x^m\} \\ \mathcal{C}_2 = \{x^{m+1}, x^{m+2}, \dots, x^N\} \end{array} \right.$

- Extended inputs $\left\{ \begin{array}{l} \mathcal{C}_1 = \{y^1, y^2, \dots, y^m\} \\ \mathcal{C}_2 = \{y^{m+1}, y^{m+2}, \dots, y^N\} \end{array} \right.$

- Find an **extended** weight vector W_* , if possible, such that :

- $\left\{ \begin{array}{lll} W_*^T y > 0 & o = 1 & y \in \mathcal{C}_1 \\ W_*^T y < 0 & o = -1 & y \in \mathcal{C}_2 \end{array} \right.$

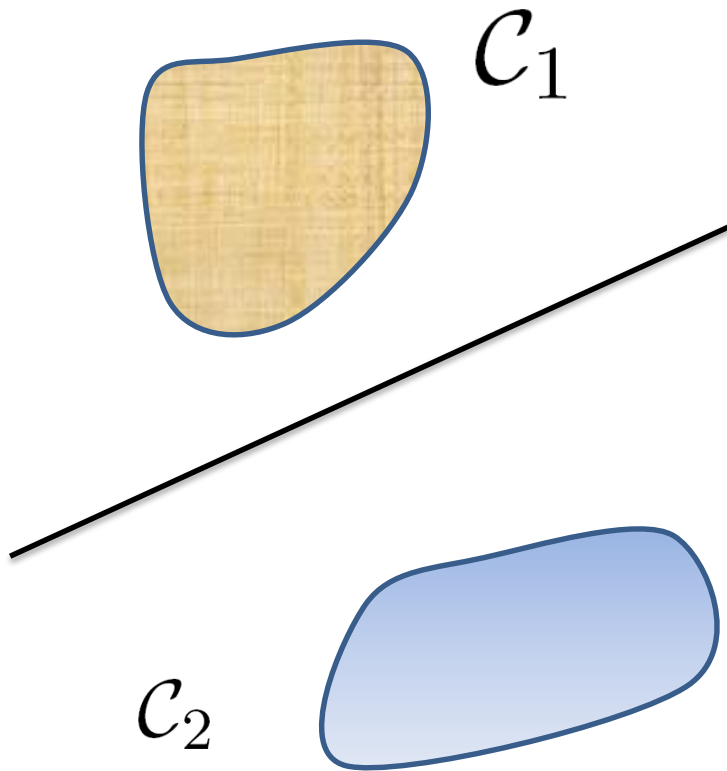
$$W^* = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ \theta \end{pmatrix}$$



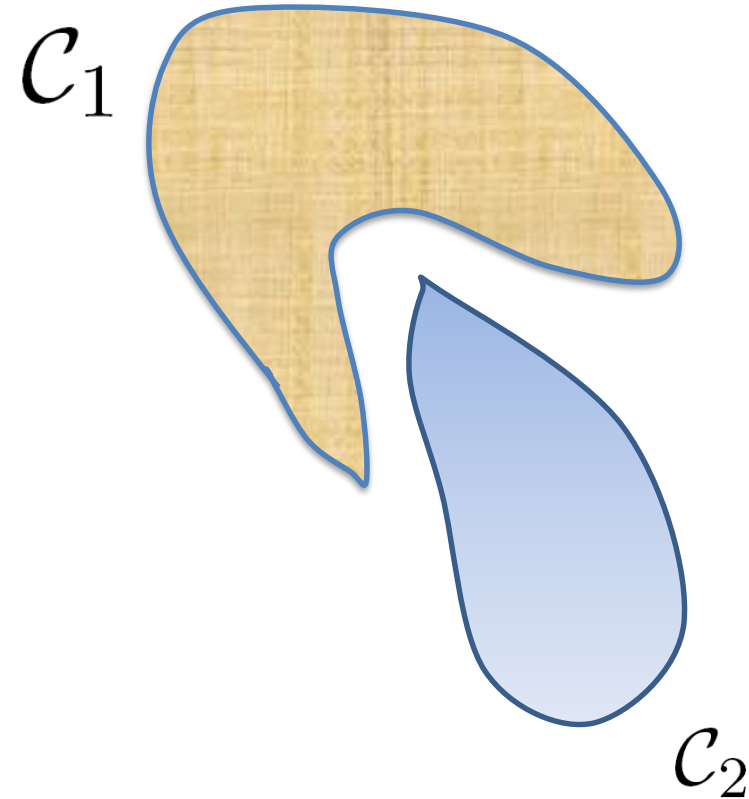
Solution (if exists!)

Corresponding single neuron

- An Obvious **necessary** condition : **LINEAR SEPARABILITY**



Line Separation is possible !



Line separation IS NOT possible

- Is this necessary condition also SUFFICIENT?
- If yes, how to find a solution?

- Is this necessary condition also SUFFICIENT?
- If yes, how to find a solution?

- **PERCEPTRON TRAINING ALGORITHM**

•
•
•

- Current weight is $W(n)$
- pick a new (extended) pattern y^i

- Is this necessary condition also SUFFICIENT?
- If yes, how to find a solution?

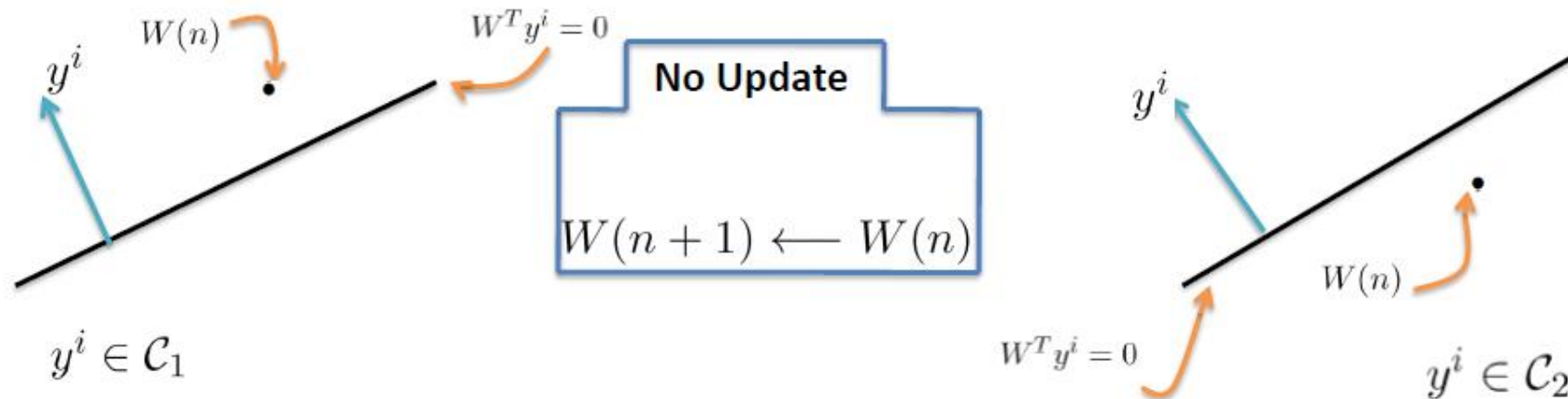
• PERCEPTRON TRAINING ALGORITHM

⋮

- Current weight is $W(n)$
- pick a new (extended) pattern y^i

3 possible cases :

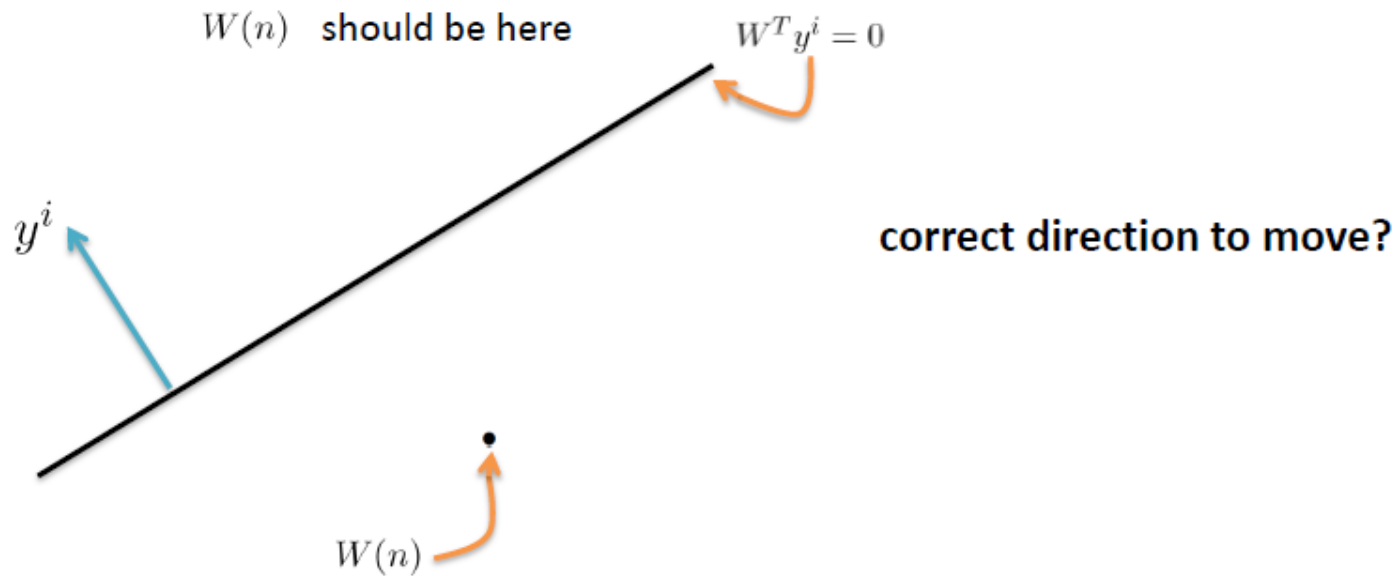
- **Case 1 : Correct Classification :** $\text{sgn}(W(n)^T y^i)$ is correct



- **Case 2 : Incorrect Classification :** $\text{sgn}(W(n)^T y^i)$ is NOT correct

- **Case 2i :** $y^i \in \mathcal{C}_1 \Rightarrow$ should have $W(n)^T y^i > 0$

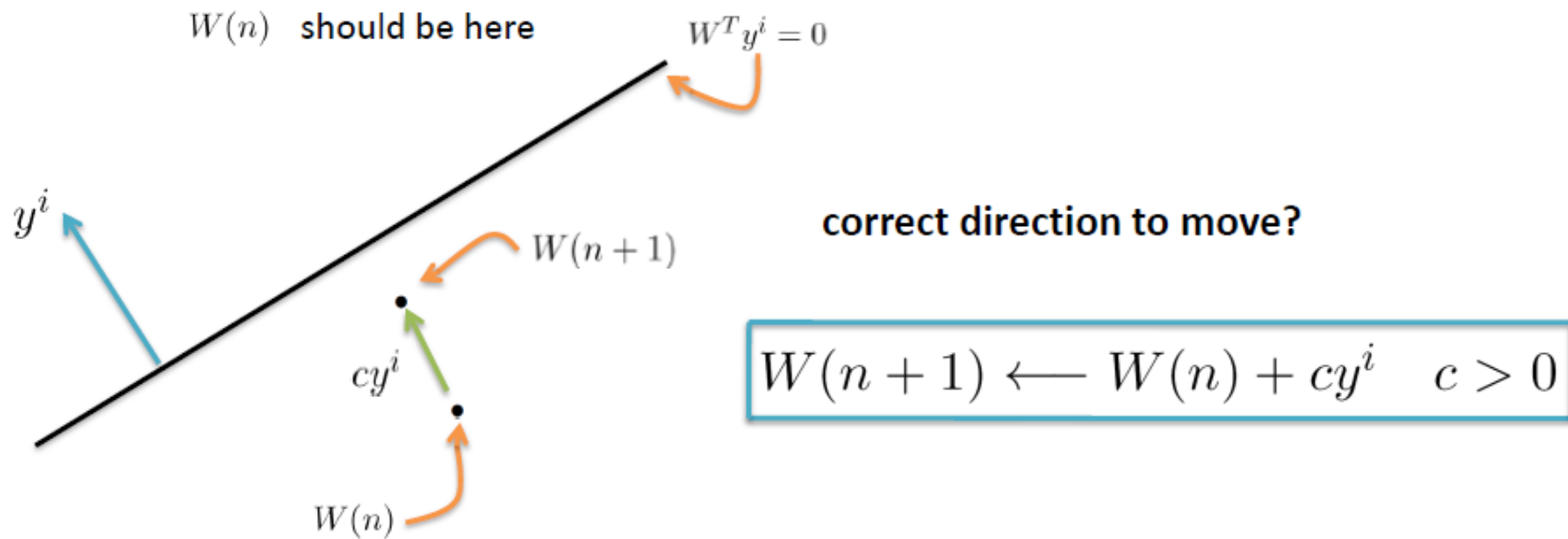
but we have $W(n)^T y^i < 0$



- **Case 2 : Incorrect Classification :** $\text{sgn}(W(n)^T y^i)$ is NOT correct

- **Case 2i :** $y^i \in \mathcal{C}_1 \Rightarrow$ should have $W(n)^T y^i > 0$

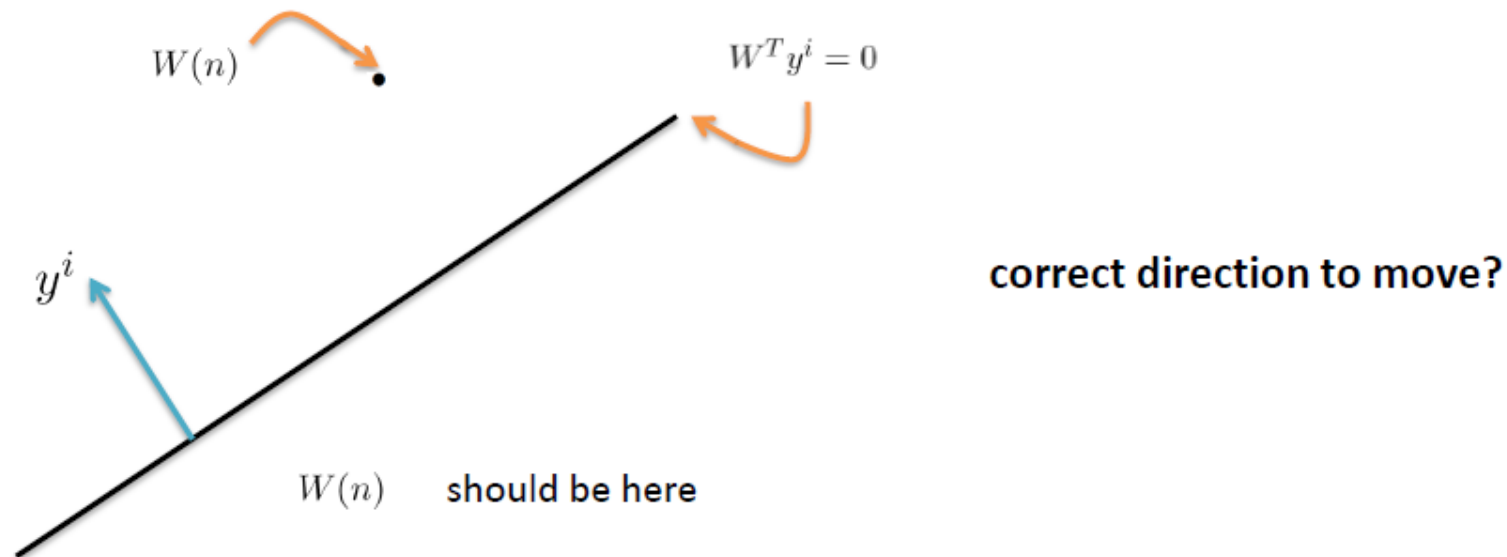
but we have $W(n)^T y^i < 0$



- **Case 2 : Incorrect Classification :** $\text{sgn}(W(n)^T y^i)$ is NOT correct

- **Case 2ii :** $y^i \in \mathcal{C}_2 \Rightarrow$ should have $W(n)^T y^i < 0$

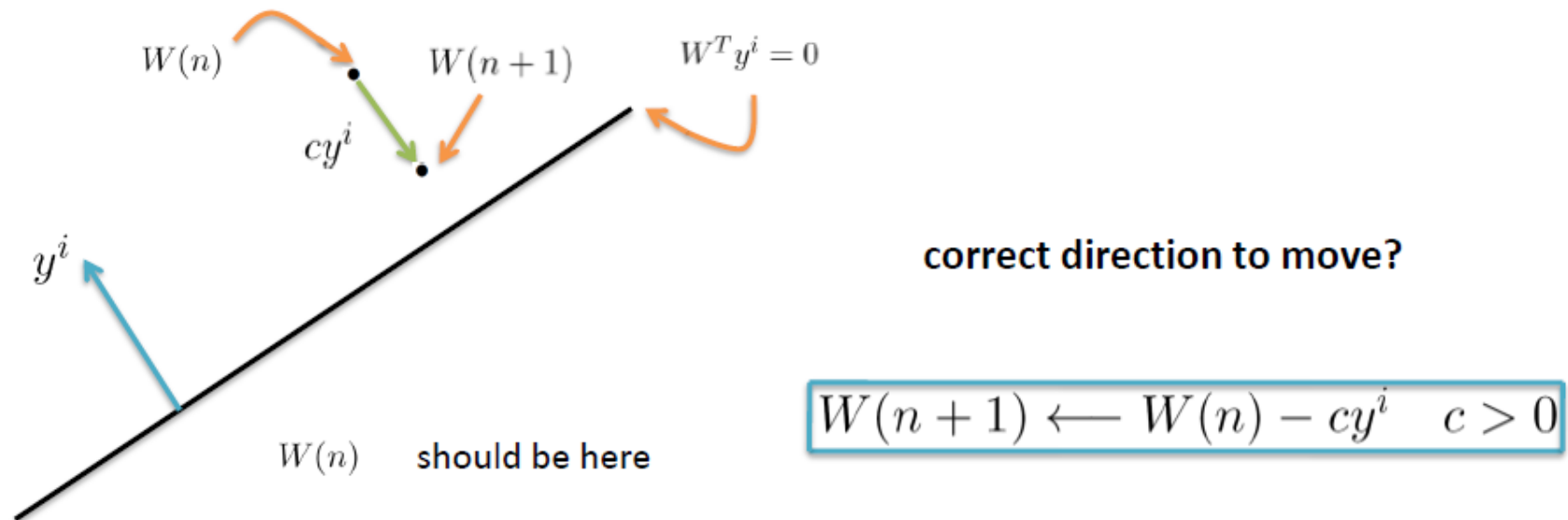
but we have $W(n)^T y^i > 0$



- **Case 2 : Incorrect Classification :** $\text{sgn}(W(n)^T y^i)$ is NOT correct

- **Case 2ii :** $y^i \in \mathcal{C}_2 \Rightarrow$ should have $W(n)^T y^i < 0$

but we have $W(n)^T y^i > 0$



Summary :

- Choose an initial weight $W(1)$
- pick a pattern, say y^1

$\vdots$

- Current weight is $W(n)$
- pick a new (extended) pattern y^i
- compute $W(n)^T y^i$

- if $\text{sgn}(W(n)^T y^i)$ is correct

$$W(n+1) \leftarrow W(n)$$

- if $y^i \in \mathcal{C}_1$ but we have $W(n)^T y^i < 0$

$$W(n+1) \leftarrow W(n) + cy^i \quad c > 0$$

- if $y^i \in \mathcal{C}_2$ but we have $W(n)^T y^i > 0$

$$W(n+1) \leftarrow W(n) - cy^i \quad c > 0$$

$\vdots$

Set $W(1)=W(N+1)$

one
epoch

- **A Compact Formulation**

- Given two classes
$$\left\{ \begin{array}{l} \mathcal{C}_1 = \{y^1, y^2, \dots, y^m\} \\ \mathcal{C}_2 = \{y^{m+1}, y^{m+2}, \dots, y^N\} \end{array} \right.$$

- For an input y^i define **desired** (**correct**) output as :

$$\bullet \left\{ \begin{array}{ll} d(i) = 1 & y^i \in \mathcal{C}_1 \\ d(i) = -1 & y^i \in \mathcal{C}_2 \end{array} \right.$$

- Current weight : $W(n)$

- Pick a pattern y^i  Compute $o(i) = \text{sign}(W(n)^T y^i)$

- Current weight : $W(n)$
- Pick a pattern y^i  Compute $o(i) = \text{sign}(W(n)^T y^i)$

$$\bullet \Delta W(i) = \begin{cases} cy^i & y^i \in \mathcal{C}_1, o(i) = -1 \\ -cy^i & y^i \in \mathcal{C}_2, o(i) = 1 \\ 0 & otherwise \end{cases}$$

- Current weight : $W(n)$
- Pick a pattern y^i  Compute $o(i) = \text{sign}(W(n)^T y^i)$

$$\bullet \Delta W(i) = \begin{cases} cy^i & y^i \in \mathcal{C}_1, o(i) = -1 \\ -cy^i & y^i \in \mathcal{C}_2, o(i) = 1 \\ 0 & otherwise \end{cases}$$

$$\bullet \Delta W(i) = \begin{cases} cy^i & d(i) = 1, o(i) = -1 \\ -cy^i & d(i) = -1, o(i) = 1 \\ 0 & otherwise \end{cases}$$

- Current weight : $W(n)$
- Pick a pattern y^i  Compute $o(i) = \text{sign}(W(n)^T y^i)$

$$\bullet \Delta W(i) = \begin{cases} cy^i & y^i \in \mathcal{C}_1, o(i) = -1 \\ -cy^i & y^i \in \mathcal{C}_2, o(i) = 1 \\ 0 & \text{otherwise} \end{cases}$$

$$\bullet \Delta W(i) = \begin{cases} cy^i & d(i) = 1, o(i) = -1 \\ -cy^i & d(i) = -1, o(i) = 1 \\ 0 & \text{otherwise} \end{cases}$$

$$\bullet \Delta W(i) = 0.5c(d(i) - o(i))y^i$$

- **Perceptron Algorithm**

- Choose an initial weight $W(1)$

- pick a pattern, say y^1
⋮

- Current Weight : $W(n)$

- pick a new (extended) pattern y^i

- Compute $o(i) = \text{sign}(W(n)^T y^i)$

- $W(n+1) \leftarrow W(n) + 0.5c(d(i) - o(i))y^i$
⋮

- Last updated weight : $W(N+1)$

- Does it solve the problem ?

one
epoch

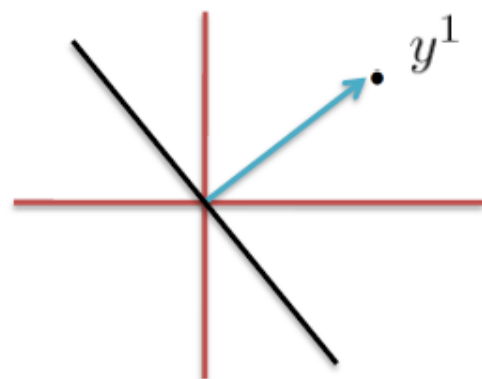
NO

YES

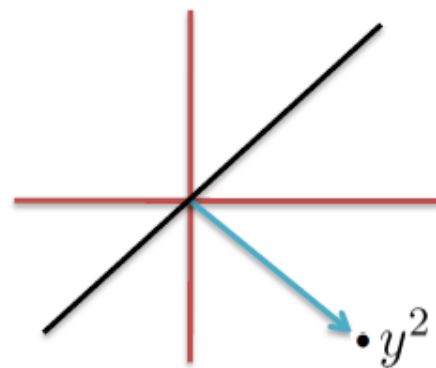
STOP

$W(1)=W(N+1)$

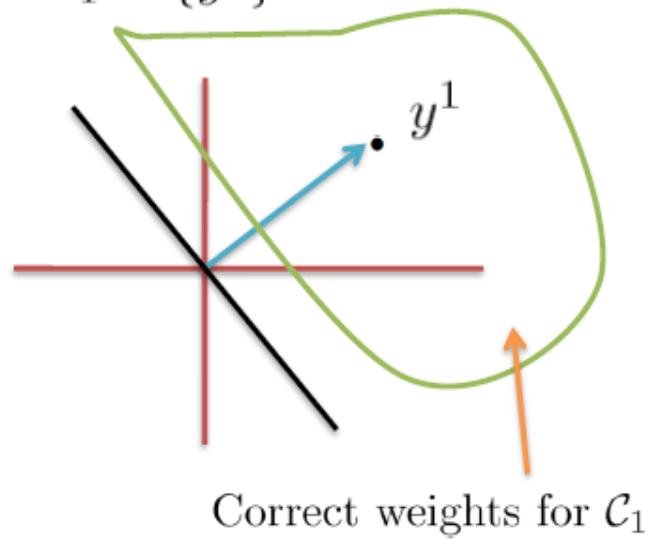
$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$



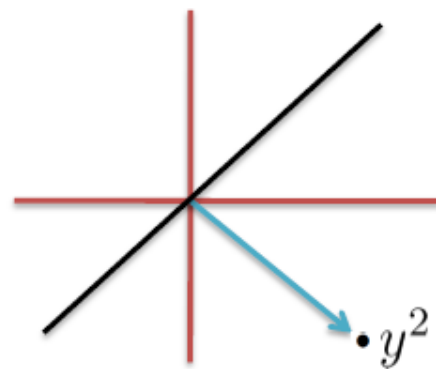
$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$



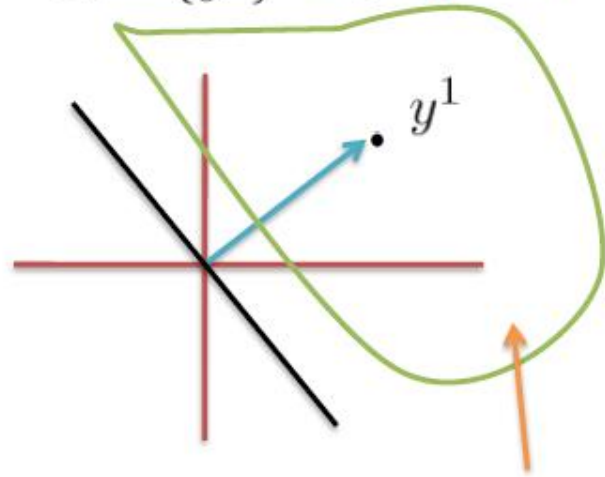
$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$



$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$

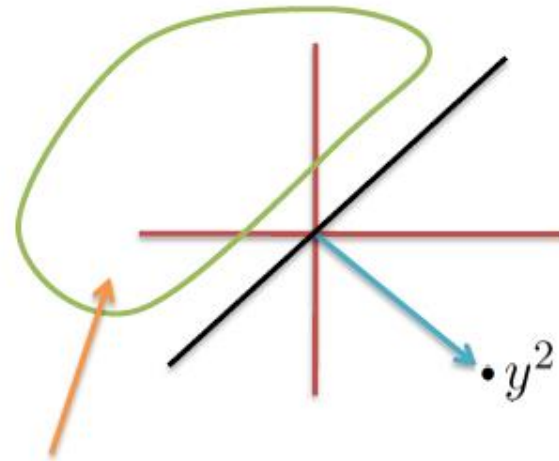


$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$



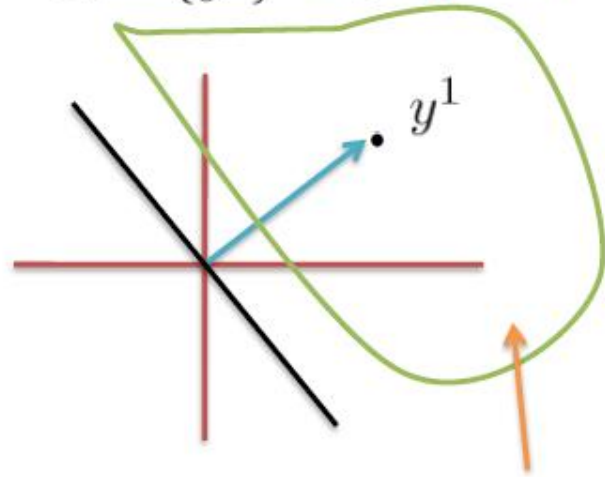
Correct weights for $\mathcal{C}_1$

$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$

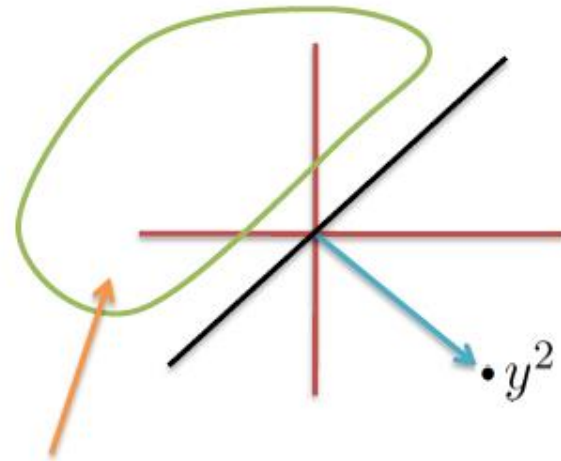


Correct weights for $\mathcal{C}_2$

$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

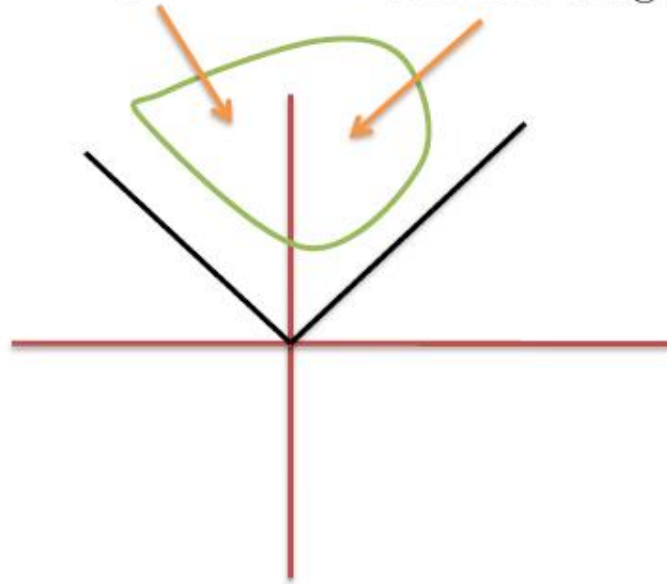


$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$



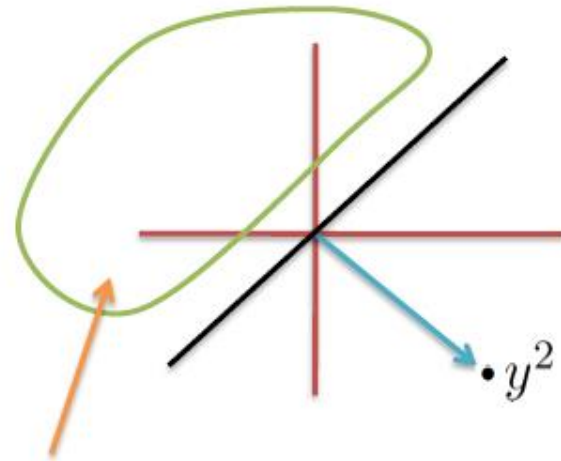
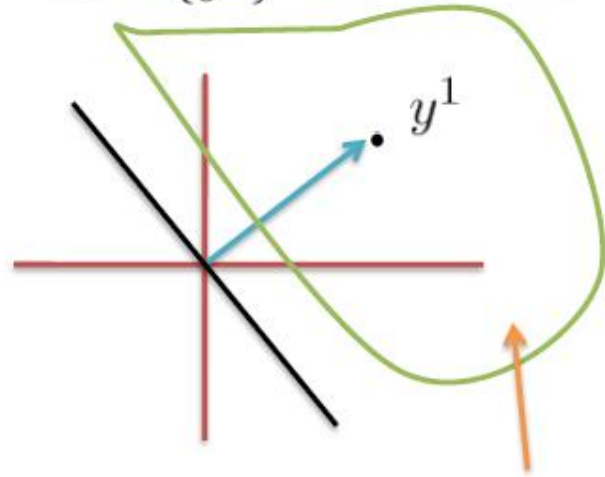
Correct weights for $\mathcal{C}_1$

Correct weights for $\mathcal{C}_2$



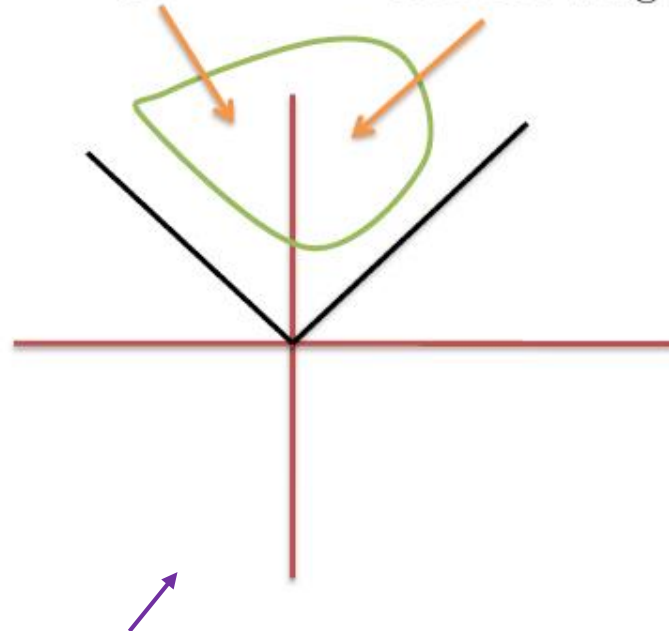
$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$



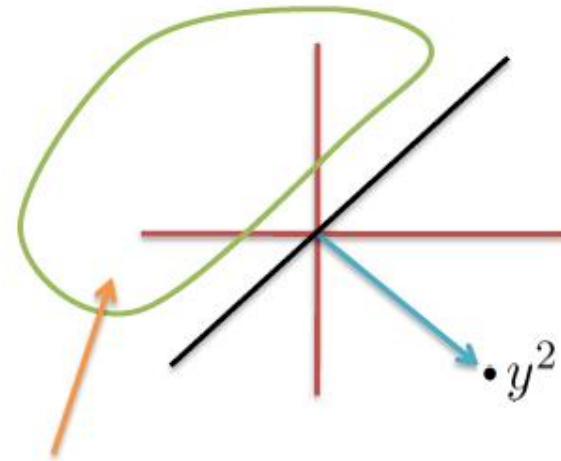
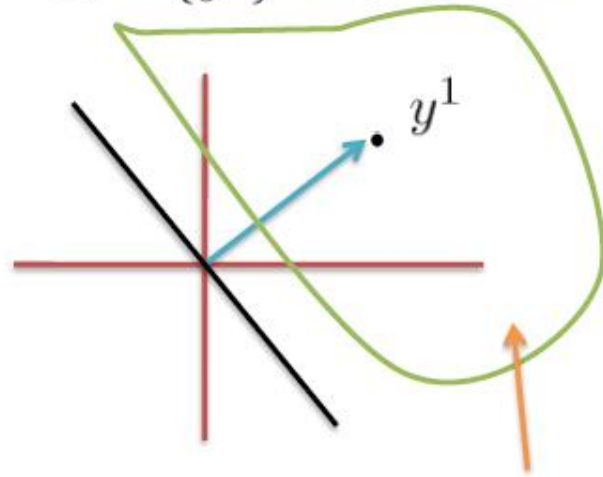
Correct weights for $\mathcal{C}_1$

Correct weights for $\mathcal{C}_2$



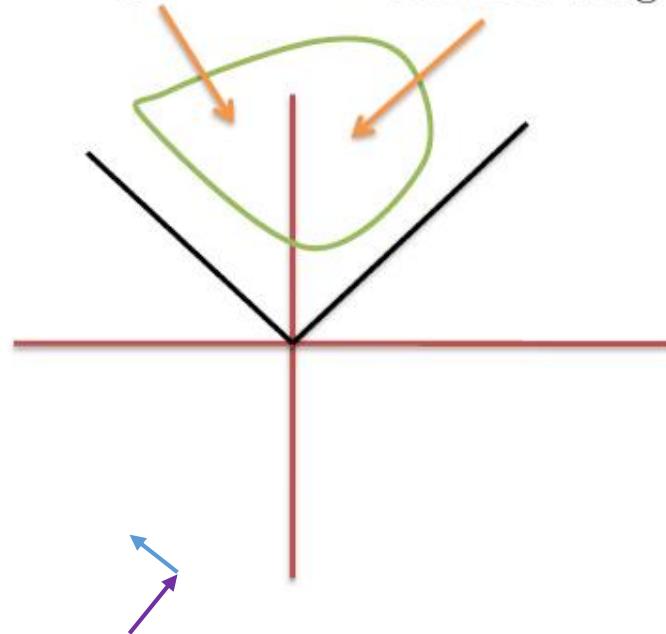
$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$



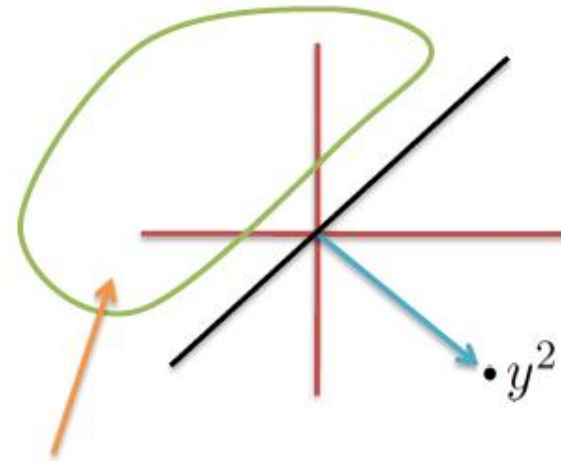
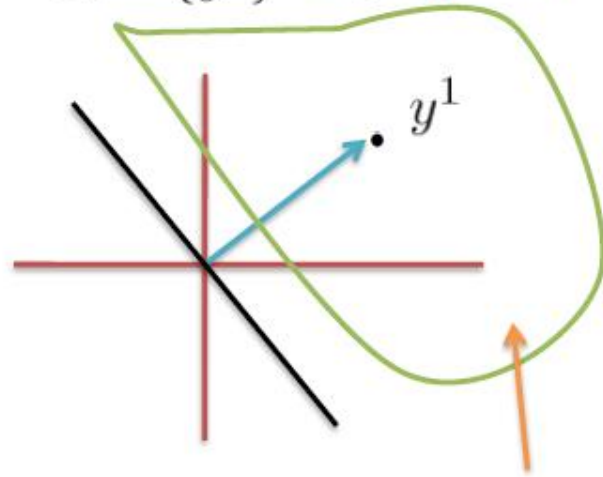
Correct weights for $\mathcal{C}_1$

Correct weights for $\mathcal{C}_2$



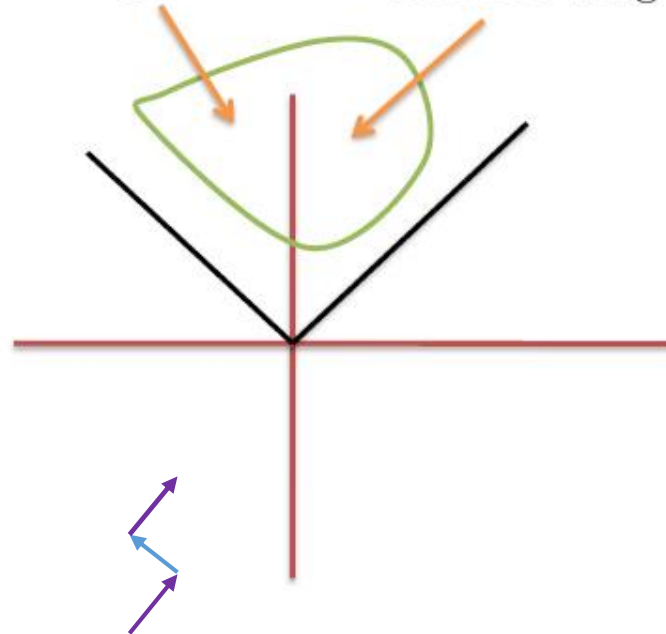
$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$



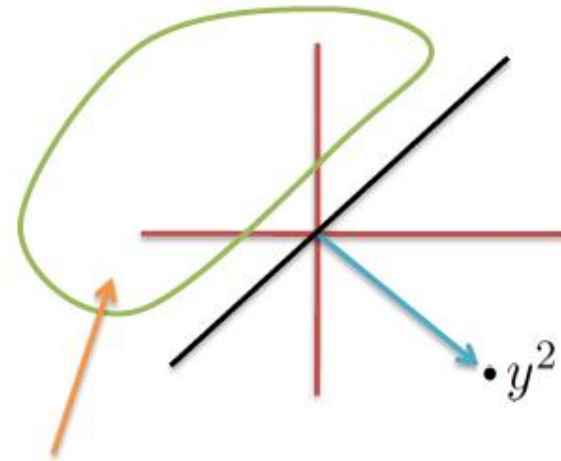
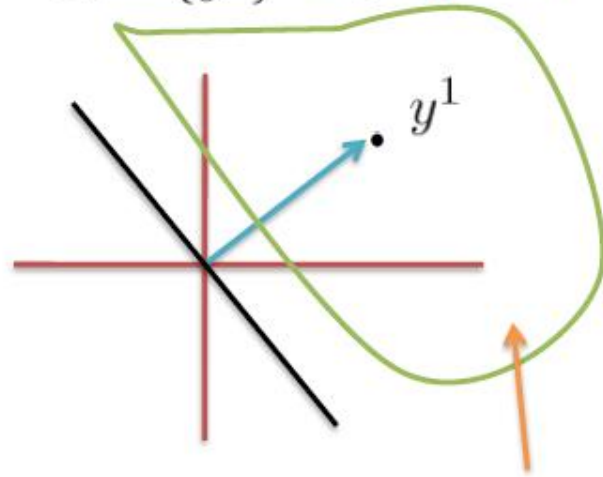
Correct weights for $\mathcal{C}_1$

Correct weights for $\mathcal{C}_2$



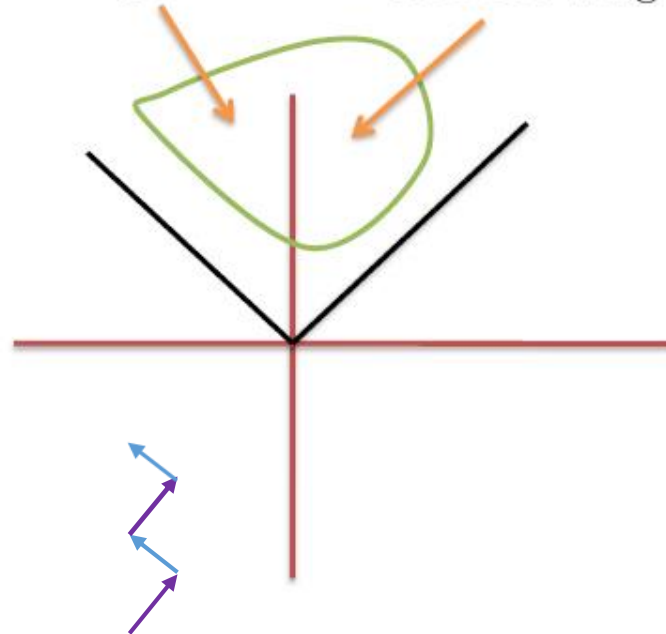
$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$

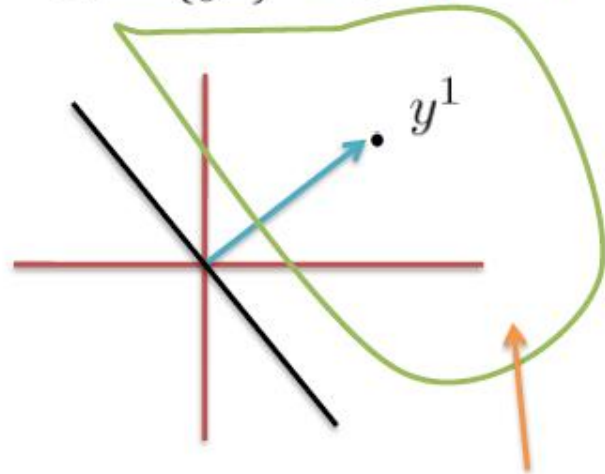


Correct weights for $\mathcal{C}_1$

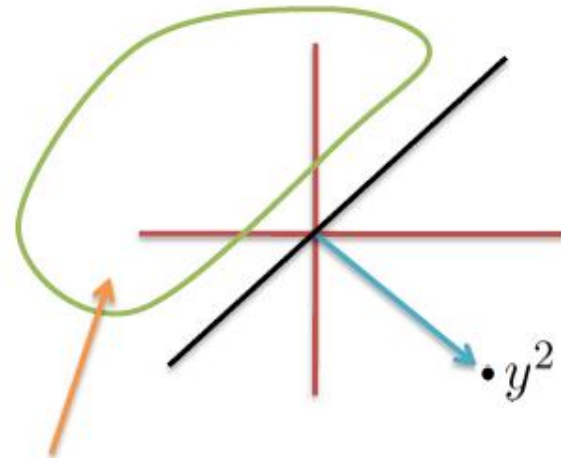
Correct weights for $\mathcal{C}_2$



$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

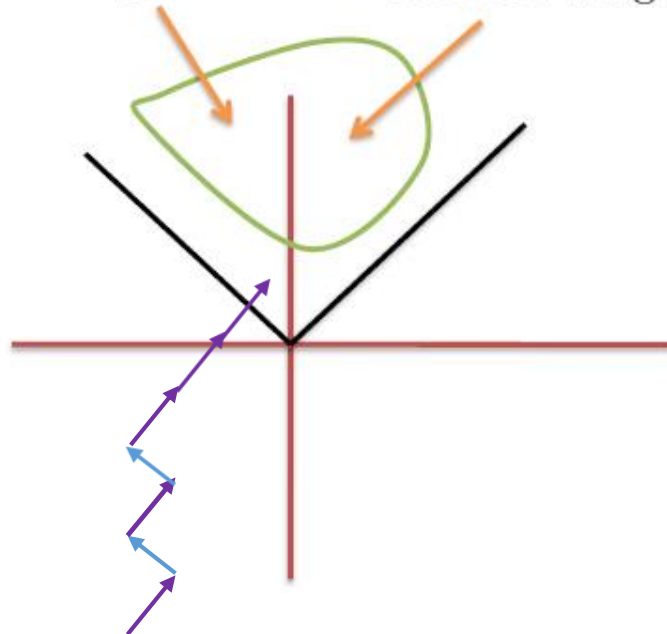


$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$

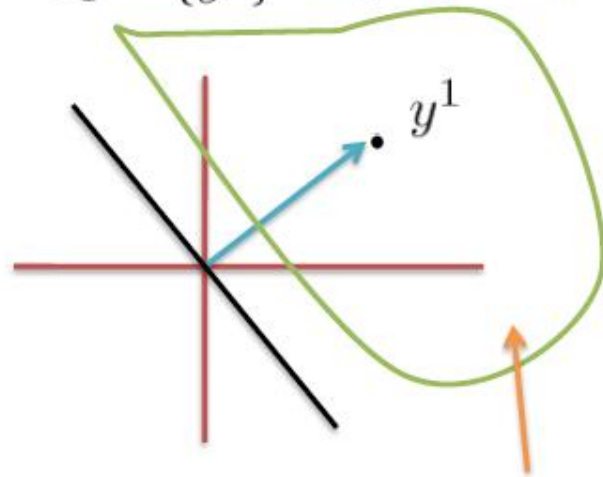


Correct weights for $\mathcal{C}_1$

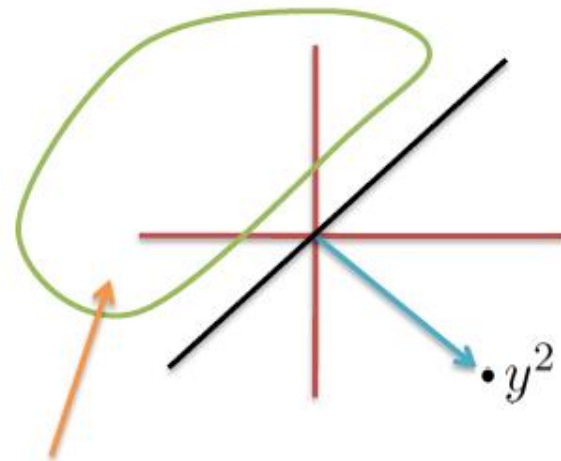
Correct weights for $\mathcal{C}_2$



$$\mathcal{C}_1 = \{y^1\} \Rightarrow o = 1$$

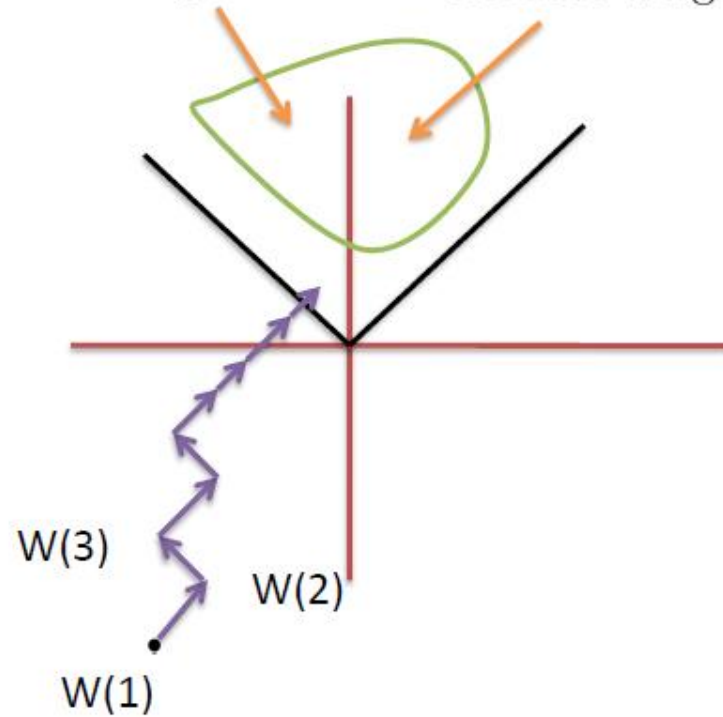


$$\mathcal{C}_2 = \{y^2\} \Rightarrow o = -1$$



Correct weights for $\mathcal{C}_1$

Correct weights for $\mathcal{C}_2$

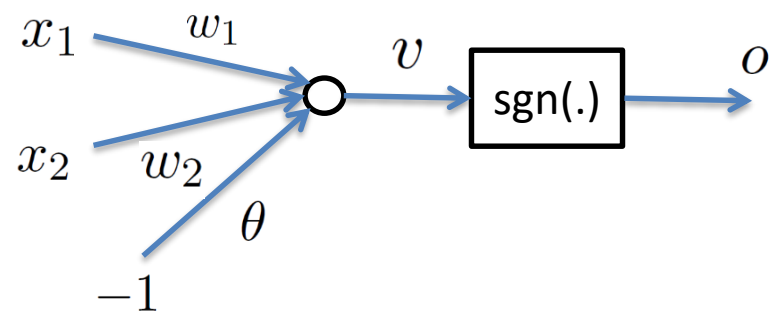
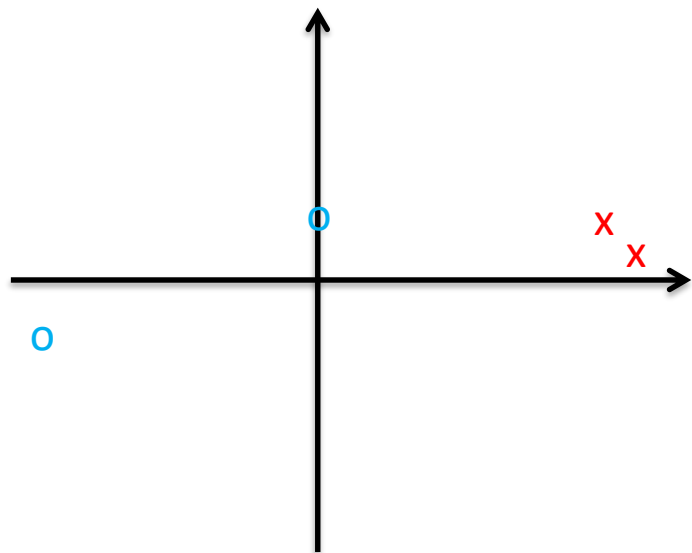


• Example

$$\mathcal{C}_1 = \left\{ y_1 = \begin{pmatrix} 10 \\ 2 \\ -1 \end{pmatrix}, y_2 = \begin{pmatrix} 9 \\ 3 \\ -1 \end{pmatrix} \right\} \quad \mathcal{C}_2 = \left\{ y_3 = \begin{pmatrix} 0 \\ 2 \\ -1 \end{pmatrix}, y_4 = \begin{pmatrix} -9 \\ -2 \\ -1 \end{pmatrix} \right\}$$

x $\Rightarrow \mathcal{C}_1$

o $\Rightarrow \mathcal{C}_2$



- Choose $c = 1$, $W(1) = \begin{pmatrix} -1 \\ -1 \\ -1 \end{pmatrix}$

• **Epoch 1 :**

$$v(1) = W(1)^T y_1 = -11 < 0 \rightarrow o(1) = -1 \rightarrow W(2) = W(1) + y_1 = \begin{pmatrix} 9 \\ 1 \\ -2 \end{pmatrix}$$

$$v(2) = W(2)^T y_2 = 86 > 0 \rightarrow o(1) = 1 \rightarrow W(3) = W(2)$$

$$v(3) = W(3)^T y_3 = 4 > 0 \rightarrow o(1) = 1 \rightarrow W(4) = W(3) - y_3 = \begin{pmatrix} 9 \\ -1 \\ -1 \end{pmatrix}$$

$$v(4) = W(4)^T y_4 = -78 < 0 \rightarrow o(1) = -1 \rightarrow W(5) = W(4)$$

• **end of Epoch 1 :**

● Epoch 2 :

$$v(5) = W(5)^T y_1 = 89 > 0 \rightarrow o(1) = 1 \rightarrow W(6) = W(5)$$

$$v(6) = W(6)^T y_2 = 79 > 0 \rightarrow o(1) = 1 \rightarrow W(7) = W(6)$$

$$v(7) = W(7)^T y_3 = -1 < 0 \rightarrow o(1) = -1 \rightarrow W(8) = W(7)$$

$$v(8) = W(8)^T y_4 = -78 < 0 \rightarrow o(1) = -1 \rightarrow W(9) = W(8)$$

● end of Epoch 2 : **STOP !**

● Epoch 2 :

$$v(5) = W(5)^T y_1 = 89 > 0 \rightarrow o(1) = 1 \rightarrow W(6) = W(5)$$

$$v(6) = W(6)^T y_2 = 79 > 0 \rightarrow o(1) = 1 \rightarrow W(7) = W(6)$$

$$v(7) = W(7)^T y_3 = -1 < 0 \rightarrow o(1) = -1 \rightarrow W(8) = W(7)$$

$$v(8) = W(8)^T y_4 = -78 < 0 \rightarrow o(1) = -1 \rightarrow W(9) = W(8)$$

● end of Epoch 2 :


$$\begin{pmatrix} 9 \\ -1 \\ -1 \end{pmatrix}$$

● Epoch 2 :

$$v(5) = W(5)^T y_1 = 89 > 0 \rightarrow o(1) = 1 \rightarrow W(6) = W(5)$$

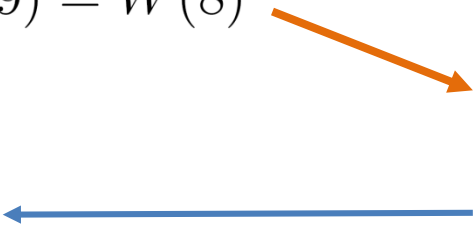
$$v(6) = W(6)^T y_2 = 79 > 0 \rightarrow o(1) = 1 \rightarrow W(7) = W(6)$$

$$v(7) = W(7)^T y_3 = -1 < 0 \rightarrow o(1) = -1 \rightarrow W(8) = W(7)$$

$$v(8) = W(8)^T y_4 = -78 < 0 \rightarrow o(1) = -1 \rightarrow W(9) = W(8)$$

● end of Epoch 2 :

● Separating line : $9x_1 - x_2 + 1 = 0$


$$\begin{pmatrix} 9 \\ -1 \\ -1 \end{pmatrix}$$

• Epoch 2 :

$$v(5) = W(5)^T y_1 = 89 > 0 \rightarrow o(1) = 1 \rightarrow W(6) = W(5)$$

$$v(6) = W(6)^T y_2 = 79 > 0 \rightarrow o(1) = 1 \rightarrow W(7) = W(6)$$

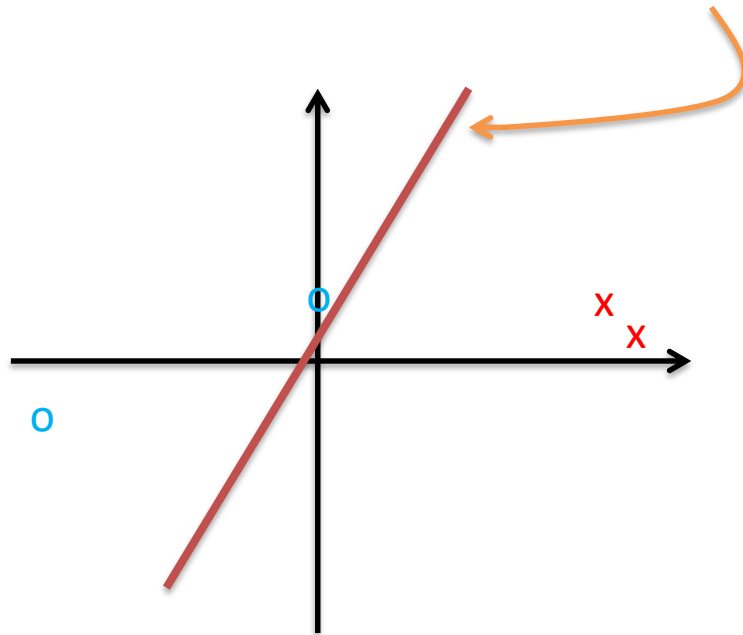
$$v(7) = W(7)^T y_3 = -1 < 0 \rightarrow o(1) = -1 \rightarrow W(8) = W(7)$$

$$v(8) = W(8)^T y_4 = -78 < 0 \rightarrow o(1) = -1 \rightarrow W(9) = W(8)$$

• end of Epoch 2 :

- Separating line : $9x_1 - x_2 + 1 = 0$

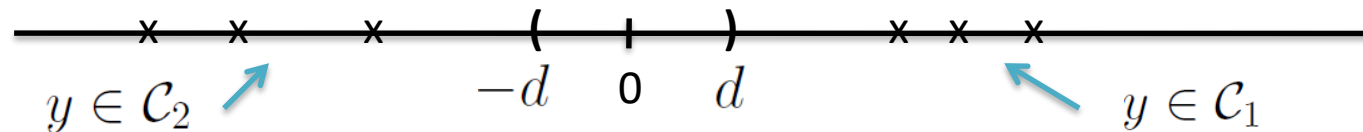
$$\begin{pmatrix} 9 \\ -1 \\ -1 \end{pmatrix}$$



• **Perceptron Convergence Theorem.** Let two classes be given as $\mathcal{C}_1 = \{y^1, y^2, \dots, y^m\}$, $\mathcal{C}_2 = \{y^{m+1}, y^{m+2}, \dots, y^n\}$ and without loss of generality, let us assume that $y \in \mathcal{C}_1$ if $W^T y > 0$ ($o = +1$), and $y \in \mathcal{C}_2$ if $W^T y < 0$ ($o = -1$). Assume that the classes are **linearly separable**. Then the perceptron algorithm given above always converge to a correct weight vector in a finite step.

• **Proof : Linear Separability** $\Rightarrow$ for some W_* we have $\begin{cases} W_*^T y > 0 \text{ if } y \in \mathcal{C}_1 \\ W_*^T y < 0 \text{ if } y \in \mathcal{C}_2 \end{cases}$

- Hence there exists a $d > 0$ such that :
- $W_*^T y > d$ if $y \in \mathcal{C}_1$, and $W_*^T y < -d$ if $y \in \mathcal{C}_2$



- **Two possible updates :**

- **Case 2i :** $y_i \in \mathcal{C}_1$ but $W(k)^T y_i < 0$

- **1 :** $W(k+1) = W(k) + cy_i$
- $\longrightarrow W_*^T W(k+1) = W_*^T W(k) + cW_*^T y_i > W_*^T W(k) + cd$

- **2 :** $\|W(k+1)\|^2 = W(k+1)^T W(k+1) = (W(k) + cy_i)^T (W(k) + cy_i)$
 $= \|W(k)\|^2 + 2cW(k)^T y_i + c^2 \|y_i\|^2$
- $\longrightarrow \|W(k+1)\|^2 < \|W(k)\|^2 + c^2 \|y_i\|^2$

- **Case 2ii :** $y_i \in \mathcal{C}_2$ but $W(k)^T y_i > 0$

- **1 :** $W(k+1) = W(k) - cy_i$ (Note that $W_*^T y_i < -d \Rightarrow -W_*^T y_i > d$)
- $\longrightarrow W_*^T W(k+1) = W_*^T W(k) - cW_*^T y_i > W_*^T W(k) + cd$

- **2 :** $\|W(k+1)\|^2 = W(k+1)^T W(k+1) = (W(k) - cy_i)^T (W(k) - cy_i)$
 $= \|W(k)\|^2 - 2cW(k)^T y_i + c^2 \|y_i\|^2$
- $\longrightarrow \|W(k+1)\|^2 < \|W(k)\|^2 + c^2 \|y_i\|^2$

- **SAME INEQUALITIES !**

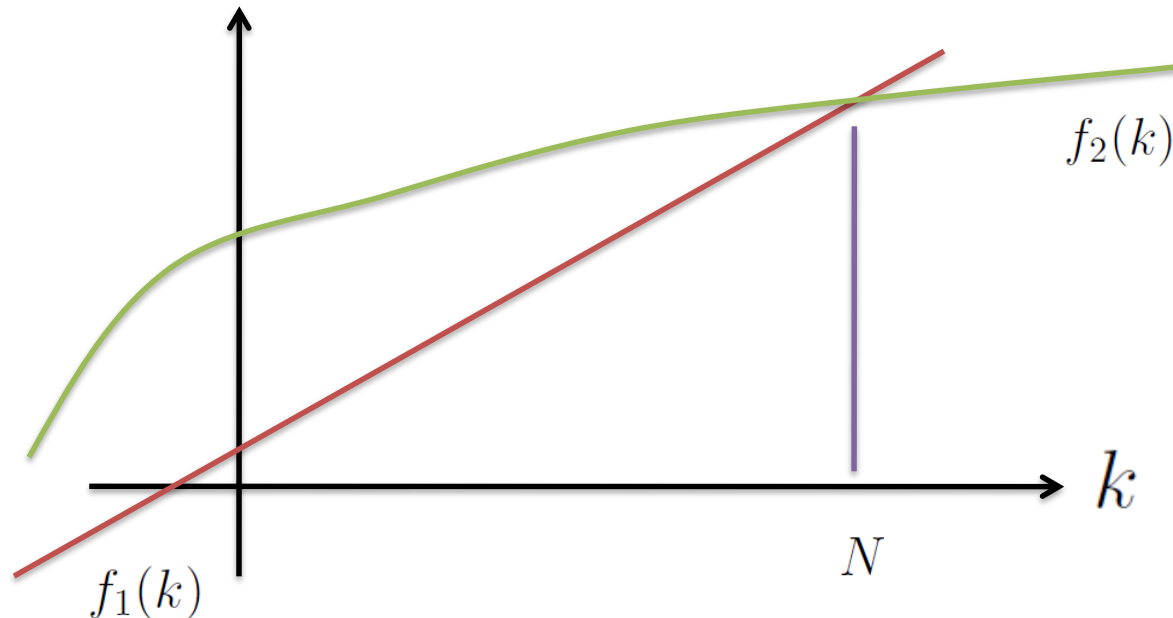
- Now let $M = \max \|y_i\|$.
- **1** : $W_*^T W(k+1) > W_*^T W(k) + cd$
- **2** : $\|W(k+1)\|^2 < \|W(k)\|^2 + c^2 \|y_i\|^2 \leq \|W(k)\|^2 + c^2 M^2$

- $W_*^T W(2) > W_*^T W(1) + cd$
- $W_*^T W(3) > W_*^T W(2) + cd > W_*^T W(1) + 2cd$
- $\vdots$
- $W_*^T W(k+1) > W_*^T W(1) + kcd$

- $\|W(2)\|^2 \leq \|W(1)\|^2 + c^2 M^2$
- $\|W(3)\|^2 \leq \|W(2)\|^2 + c^2 M^2 \leq \|W(1)\|^2 + 2c^2 M^2$
- $\vdots$
- $\|W(k+1)\|^2 \leq \|W(1)\|^2 + kc^2 M^2$

- Cauchy-Schwartz inequality $\rightarrow |x^T y| \leq \|x\| \|y\|$

- $\|W_*\| \|W(k+1)\| \geq W_*^T W(1) + kcd$
- $\|W(k+1)\| \leq \sqrt{\|W(1)\|^2 + kc^2 M^2}$
- $f_1(k) = \frac{W_*^T W(1) + kcd}{\|W_*\|} \leq \|W(k+1)\| \leq \sqrt{\|W(1)\|^2 + kc^2 M^2} = f_2(k)$



- Hence there exist an integer N such that for any iteration $k > N$, the above inequality is not satisfied $\rightarrow$ **contradiction**.

- Therefore, the algorithm stops at most after N steps.

- For an estimation, and for simplicity
assume that $\|W_*\| = 1, W(1) = 0$.

- $Ncd > \sqrt{Nc^2M^2} = cM\sqrt{N} \rightarrow N \approx M^2/d^2$

- Inversely proportional to separation



Perceptron Classifiers : Multicategory Case

given R classes $\mathcal{C}_1 = \{x^1, x^2, \dots, x^m\} \dots \mathcal{C}_R = \{x^{q+1}, x^{q+2}, \dots, x^N\}$

extended $\mathcal{C}_1 = \{y^1, y^2, \dots, y^m\} \dots \mathcal{C}_R = \{y^{q+1}, y^{q+2}, \dots, y^N\}$

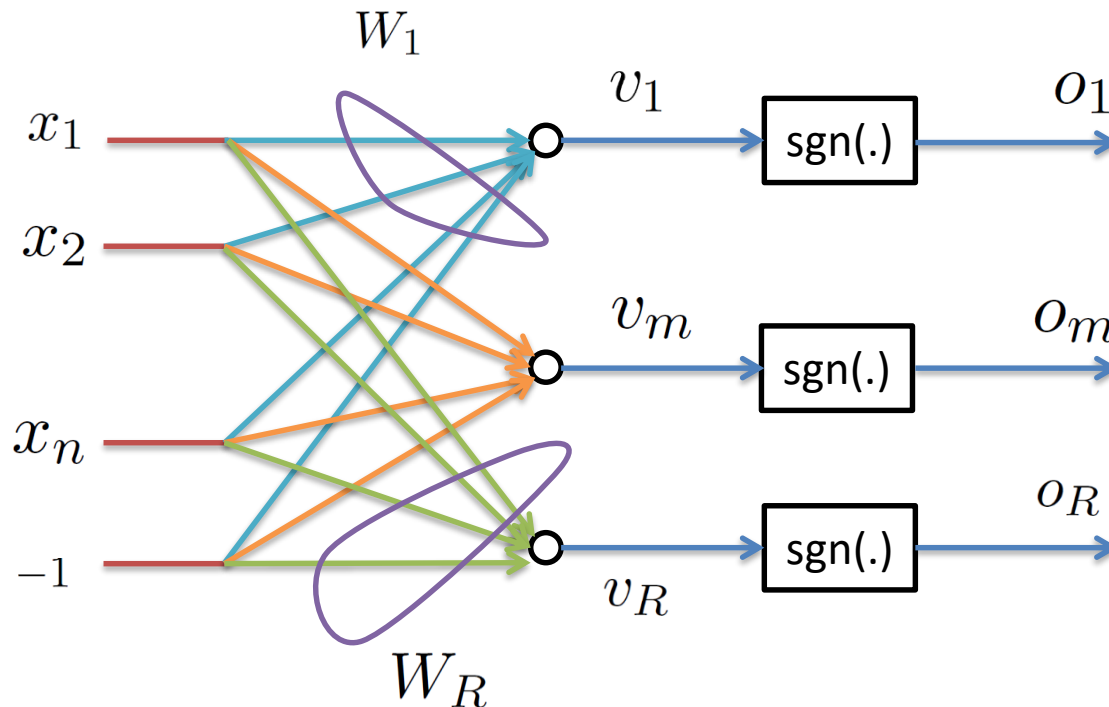
R perceptron neurons $\rightarrow R$ set of weights.

Given pattern y :

$$v_i = W_{ie}^T y \longrightarrow o_i = \text{sign}(v_i) \quad i = 1, 2, \dots, R$$

$$W_j = \begin{pmatrix} w_{j1} \\ w_{j2} \\ \vdots \\ w_{jn} \end{pmatrix}, \quad x^i = \begin{pmatrix} x_1^i \\ x_2^i \\ \vdots \\ x_n^i \end{pmatrix}, \quad W_{je} = \begin{pmatrix} w_{j1} \\ w_{j2} \\ \vdots \\ w_{jn} \\ \theta_j \end{pmatrix}, \quad x_e^i = y^i = \begin{pmatrix} x_1^i \\ x_2^i \\ \vdots \\ x_n^i \\ -1 \end{pmatrix}$$

- **Perceptron Classification Problem : Multicategory Case**
- Given R classes $\mathcal{C}_1, \dots, \mathcal{C}_R$, find R extended weights $W_{1*}, \dots, W_{R*}$
- Given any pattern y^i , compute $v_1 = W_{1*}^T y^i, \dots, v_R = W_{R*}^T y^i$
 - $y^i \in \mathcal{C}_m \iff v_m > v_j, \quad j = 1, 2, \dots, R, \quad j \neq m$



• Perceptron Training Algorithm : Multicategory case

- choose initial weights $W_1(1), \dots, W_R(1)$, $c > 0$

- last updated weights $W_1(n), \dots, W_R(n)$
- Pick a pattern y^i from the training set. Assume that $y^i \in \mathcal{C}_m$
- Compute $v_1(n) = W_1^T(n)y^i, \dots, v_R(n) = W_R^T(n)y^i$
- If $v_j(n) \geq v_m(n)$ for some $j = 1, 2, \dots, R$, $j \neq m$,
 - $W_m(n+1) = W_m(n) + cy^i$
 - $W_j(n+1) = W_j(n) - cy^i$
- Otherwise, do not update the corresponding weights.
- Choose another pattern and repeat

Last update



if solved : STOP

if NOT

- Why does it work ? similar to two class case.

Assume that for a $y^i \in \mathcal{C}_m$ we have misclassification :

→ we have $v_j(n) \geq v_m(n)$ for some $j = 1, 2, \dots, R$,

updates →
$$\begin{cases} W_m(n+1) = W_m(n) + cy^i \\ W_j(n+1) = W_j(n) - cy^i \end{cases}$$

apply y^i again :

$$\begin{aligned} v_j(n+1) &= W_j(n+1)^T y^i = [W_j(n) - cy^i]^T y^i \\ &= W_j(n)^T y^i - cy^{iT} y^i < v_j(n) \end{aligned}$$

v_j decreases

$$\begin{aligned} v_m(n+1) &= W_m(n+1)^T y^i = [W_m(n) + cy^i]^T y^i \\ &= W_m(n)^T y^i + cy^{iT} y^i > v_m(n) \end{aligned}$$



v_m increases

- This method uses the **maximum selection** based on v_j :
- **Step 1** Given pattern y , calculate $v_1, v_2, \dots, v_R$
- **Step 2** Find the index j such that $v_j = \max\{v_1, v_2, \dots, v_R\}$
- **Step 3** $y \in \mathcal{C}_j \quad \Rightarrow \quad \text{DECISION}$

Selection based on output ?

$$o = 1 \quad \longleftrightarrow \quad v \geq 0$$

$$o = -1 \quad \longleftrightarrow \quad v < 0$$

SOLUTION : for $y^i \in \mathcal{C}_m$, keep updating till

$$v_m \geq 0$$

$$v_j < 0 \quad j \neq m$$

For each input vector y^i define the **DESIRED** output vector d^i :

$$d^i = \begin{pmatrix} d_1^i \\ d_2^i \\ \vdots \\ d_R^i \end{pmatrix} \quad \Rightarrow \quad d_j^i = \begin{cases} +1 & j = m \\ -1 & j \neq m \end{cases} \quad y^i \in \mathcal{C}_m$$

$$d^1 = \begin{pmatrix} 1 \\ -1 \\ \vdots \\ -1 \end{pmatrix} \quad \begin{matrix} \text{1}^{st} \\ \swarrow \end{matrix} \quad d^2 = \begin{pmatrix} -1 \\ 1 \\ \vdots \\ -1 \end{pmatrix} \quad \begin{matrix} \text{2}^{nd} \\ \swarrow \end{matrix} \quad \dots \quad d^m = \begin{pmatrix} -1 \\ \vdots \\ 1 \\ \vdots \\ -1 \end{pmatrix} \quad \begin{matrix} m^{th} \\ \swarrow \end{matrix} \quad \dots \quad d^R = \begin{pmatrix} -1 \\ -1 \\ \vdots \\ 1 \end{pmatrix} \quad \begin{matrix} R^{th} \\ \swarrow \end{matrix}$$

$\mathcal{C}_1 \qquad \mathcal{C}_2 \qquad \mathcal{C}_m \qquad \mathcal{C}_R$

• Perceptron Training Algorithm : Multicategory case

- Choose initial weights $W_1(1), \dots, W_R(1)$ and $c > 0$

- Pick a pattern, say y^1
 $\vdots$

- Last updated weights : $W_1(n), \dots, W_R(n)$

- Compute $v_1(n) = W_1(n)y^i, \dots, v_R(n) = W_R(n)y^i$

Compute $o_1(n) = \text{sgn}(v_1(n)), \dots, o_R(n) = \text{sgn}(v_R(n))$

$$W_j(n+1) = W_j(n) + 0.5c (d_j^i - o_j(n))y^i, \quad j = 1, 2, \dots, R$$

$\vdots$

- Repeat till last pattern....

- Last updated weights : $W_1(N+1), \dots, W_R(N+1)$

- Is the problem solved ?

NO



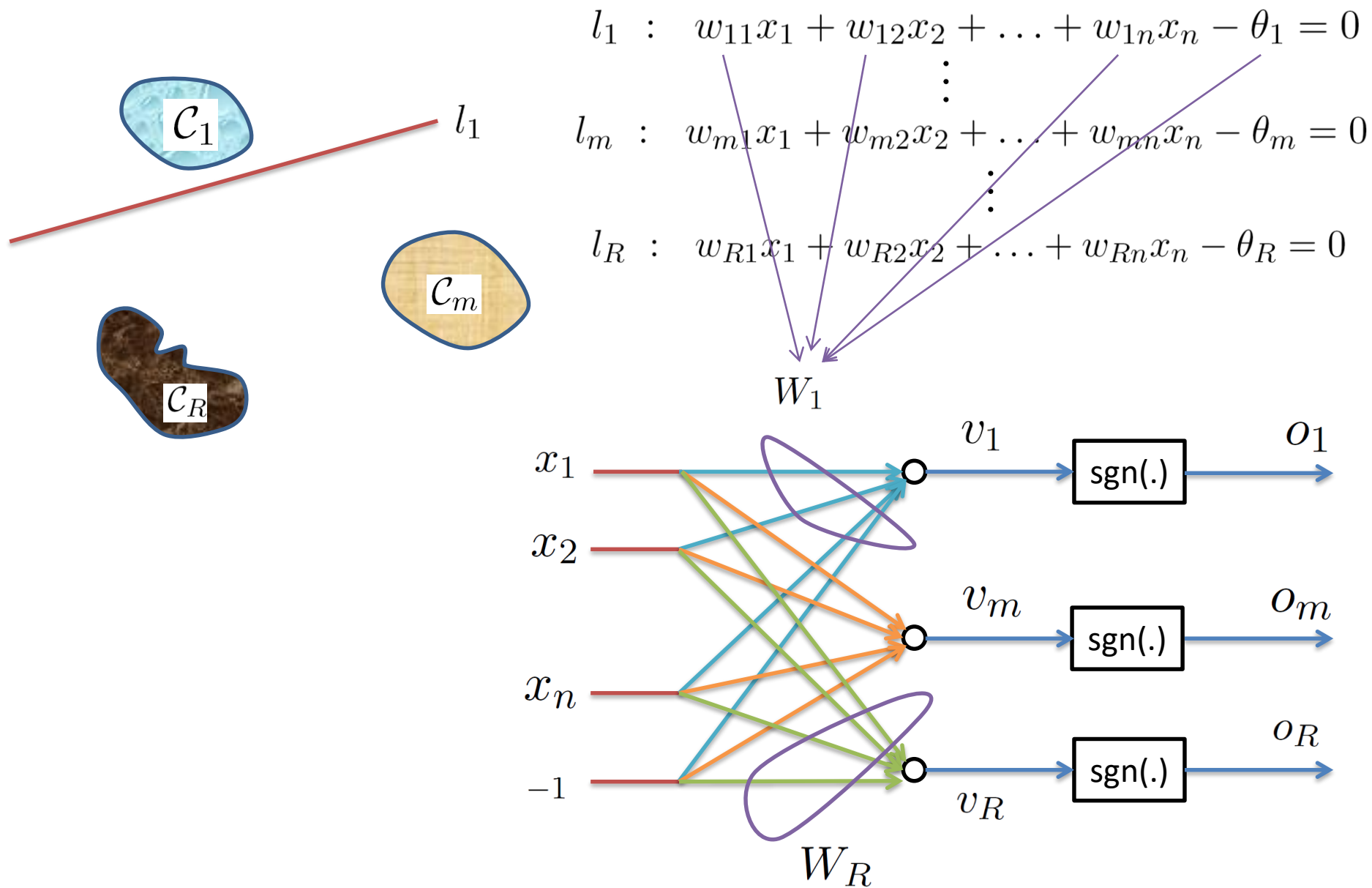
YES

STOP

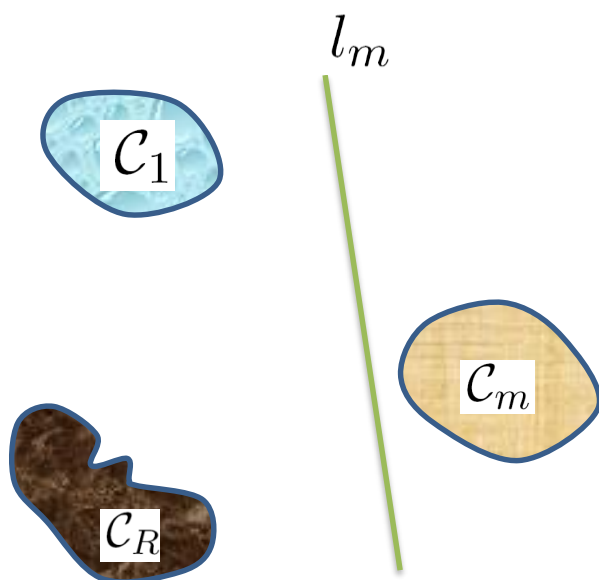
$$W_1(1) = W_1(N+1), \dots, W_R(1) = W_R(N+1)$$



Geometric Interpretation

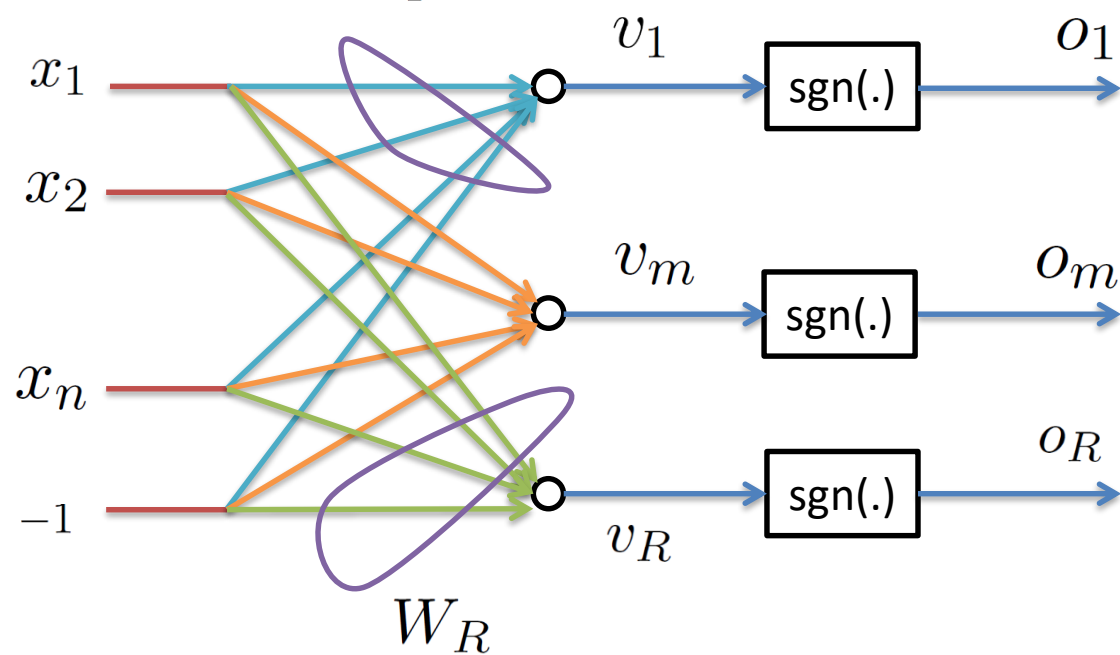


Geometric Interpretation

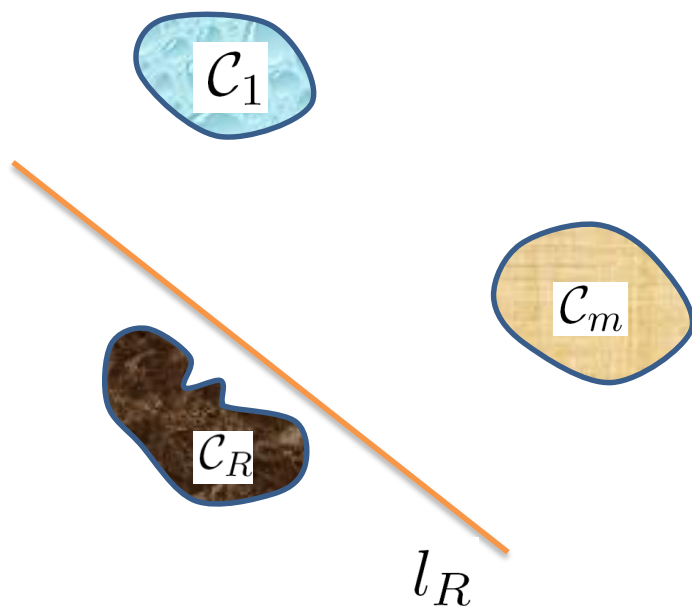


$$\begin{aligned}
 l_1 &: w_{11}x_1 + w_{12}x_2 + \dots + w_{1n}x_n - \theta_1 = 0 \\
 &\quad \vdots \\
 l_m &: w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mn}x_n - \theta_m = 0 \\
 &\quad \vdots \\
 l_R &: w_{R1}x_1 + w_{R2}x_2 + \dots + w_{Rn}x_n - \theta_R = 0
 \end{aligned}$$

Arrows from the weight terms $w_{11}, w_{12}, \dots, w_{1n}$ in the first equation point to a common point labeled W_1 . Similarly, arrows from the weight terms $w_{m1}, w_{m2}, \dots, w_{mn}$ in the m -th equation point to a common point labeled W_m . Arrows from the weight terms $w_{R1}, w_{R2}, \dots, w_{Rn}$ in the R -th equation point to a common point labeled W_R .

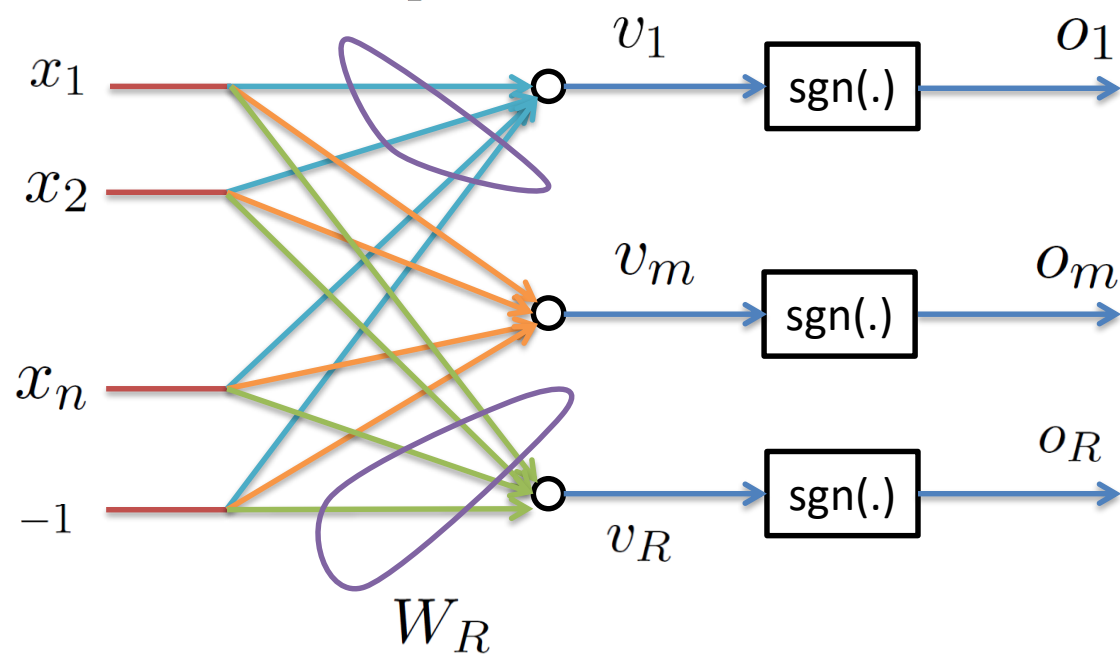


Geometric Interpretation

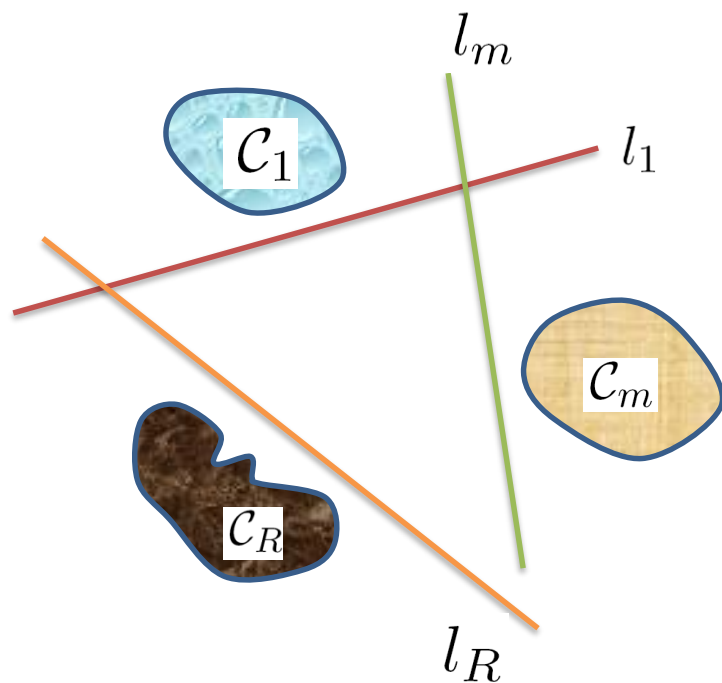


$$\begin{aligned}
 l_1 &: w_{11}x_1 + w_{12}x_2 + \dots + w_{1n}x_n - \theta_1 = 0 \\
 &\quad \vdots \\
 l_m &: w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mn}x_n - \theta_m = 0 \\
 &\quad \vdots \\
 l_R &: w_{R1}x_1 + w_{R2}x_2 + \dots + w_{Rn}x_n - \theta_R = 0
 \end{aligned}$$

Four purple arrows point from the weight terms w_{11}, w_{12}, w_{1n} in the first equation to the label W_1 . Similarly, four purple arrows point from the weight terms w_{m1}, w_{m2}, w_{mn} in the second equation to the label W_R .

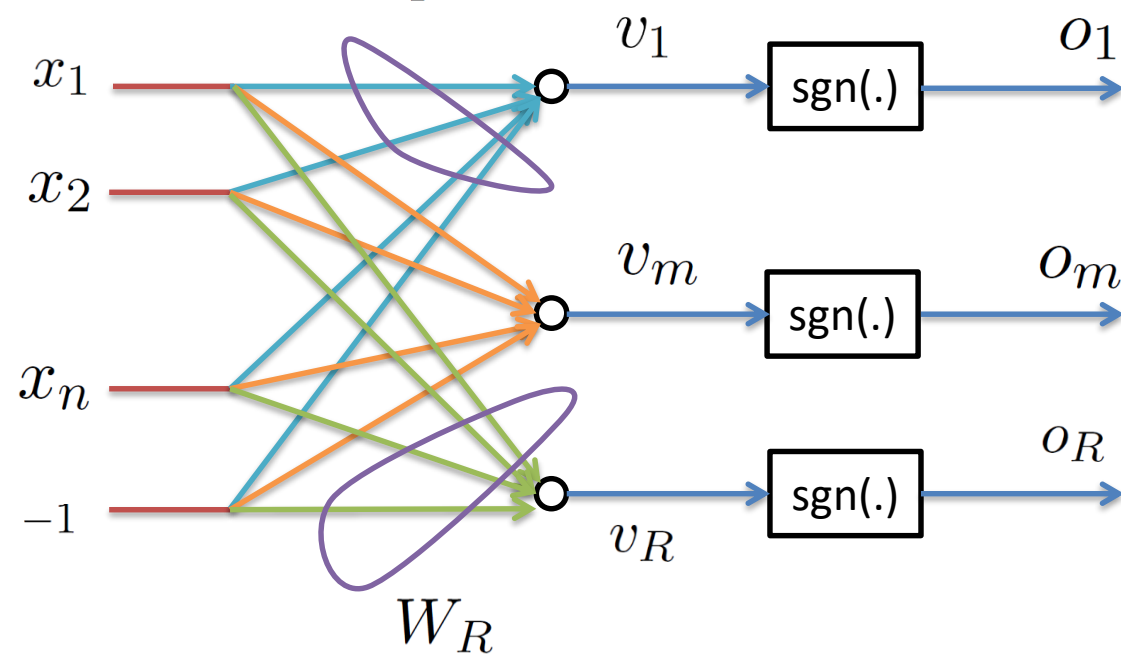


Geometric Interpretation



$$\begin{aligned}
 l_1 &: w_{11}x_1 + w_{12}x_2 + \dots + w_{1n}x_n - \theta_1 = 0 \\
 &\quad \vdots \\
 l_m &: w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mn}x_n - \theta_m = 0 \\
 &\quad \vdots \\
 l_R &: w_{R1}x_1 + w_{R2}x_2 + \dots + w_{Rn}x_n - \theta_R = 0
 \end{aligned}$$

Arrows from the weight terms $w_{11}, w_{12}, \dots, w_{1n}$ in the first equation point to a common point labeled W_1 . Similarly, arrows from the weight terms in the other equations point to a common point labeled W_R .



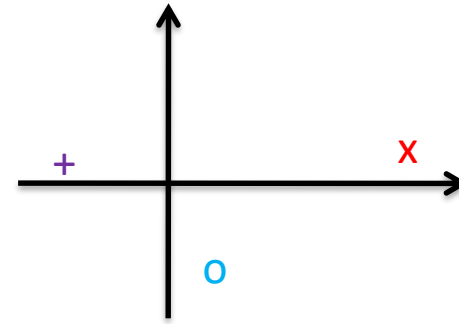
• **Perceptron Convergence Theorem : Multicategory case .** Given R classes which are linearly separable, the algorithm given above always converge in a finite number of steps.

Example : Given 3 classes C_1, C_2, C_3 as follows :

$$\bullet \underset{\text{red } \times}{C_1} = \left\{ x^1 = \begin{pmatrix} 10 \\ 2 \end{pmatrix} \right\}, \underset{\text{blue } o}{C_2} = \left\{ x^2 = \begin{pmatrix} 2 \\ -5 \end{pmatrix} \right\}, \underset{\text{purple } +}{C_3} = \left\{ x^3 = \begin{pmatrix} -5 \\ 1 \end{pmatrix} \right\}$$

$$\bullet C_1 = \left\{ y^1 = \begin{pmatrix} 10 \\ 2 \\ -1 \end{pmatrix} \right\}, C_2 = \left\{ y^2 = \begin{pmatrix} 2 \\ -5 \\ -1 \end{pmatrix} \right\}, C_3 = \left\{ y^3 = \begin{pmatrix} -5 \\ 1 \\ -1 \end{pmatrix} \right\}$$

$$\bullet d^1 = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}, d^2 = \begin{pmatrix} -1 \\ 1 \\ -1 \end{pmatrix}, d^3 = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}$$



- Initial Weights : $W_1(1) = \begin{pmatrix} 1 \\ -2 \\ 0 \end{pmatrix}$, $W_2(1) = \begin{pmatrix} -2 \\ 4 \\ 3 \end{pmatrix}$, $W_3(1) = \begin{pmatrix} 1 \\ 3 \\ -1 \end{pmatrix}$

- Choose the learning coefficient $c = 1$

Epoch 1 :

- $\mathcal{C}_1 = \left\{ y^1 = \begin{pmatrix} 10 \\ 2 \\ -1 \end{pmatrix} \right\}$, $\mathcal{C}_2 = \left\{ y^2 = \begin{pmatrix} 2 \\ -5 \\ -1 \end{pmatrix} \right\}$, $\mathcal{C}_3 = \left\{ y^3 = \begin{pmatrix} -5 \\ 1 \\ -1 \end{pmatrix} \right\}$

$$v_1 = W_1^T y^1 = 6 \quad o = 1$$

$$v_2 = W_2^T y^1 = -15 \quad o = -1$$

$$v_3 = W_3^T y^1 = 17 \quad o = 1 \quad * \Rightarrow \text{Needs update !}$$

- $W_1(2) = W_1(1)$, $W_2(2) = W_2(1)$, $W_3(2) = W_3(1) - y^1 = \begin{pmatrix} -9 \\ 1 \\ 0 \end{pmatrix}$

 Last updated

• Apply y^2 :

$$\left. \begin{aligned} v_1 &= W_1^T y^2 = 12 & o &= 1 & * \\ v_2 &= W_2^T y^2 = -27 & o &= -1 & * \\ v_3 &= W_3^T y^2 = -23 & o &= -1 \end{aligned} \right\} \text{Need Correction}$$

• $W_1(3) = W_1(2) - y^2 = \begin{pmatrix} -1 \\ 3 \\ 1 \end{pmatrix}$, $W_2(3) = W_2(2) + y^2 = \begin{pmatrix} 0 \\ -1 \\ 2 \end{pmatrix}$ $W_3(3) = W_3(2)$

• Apply y^3 :

$$v_1 = W_1^T y^3 = 7 \quad o = 1 \quad * \quad \text{Needs Correction}$$

$$v_2 = W_2^T y^3 = -3 \quad o = -1$$

$$v_3 = W_3^T y^3 = 46 \quad o = 1$$

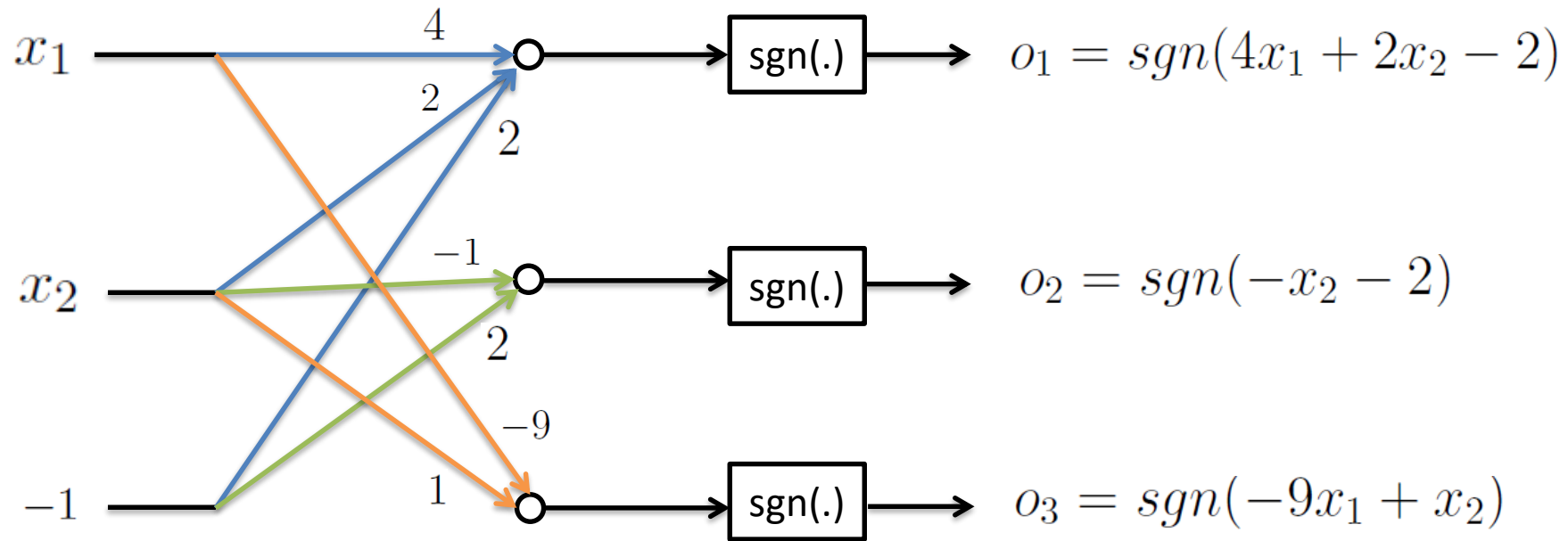
• $W_1(4) = W_1(3) - y^3 = \begin{pmatrix} 4 \\ 2 \\ 2 \end{pmatrix}$, $W_2(4) = W_2(3)$, $W_3(4) = W_3(3)$

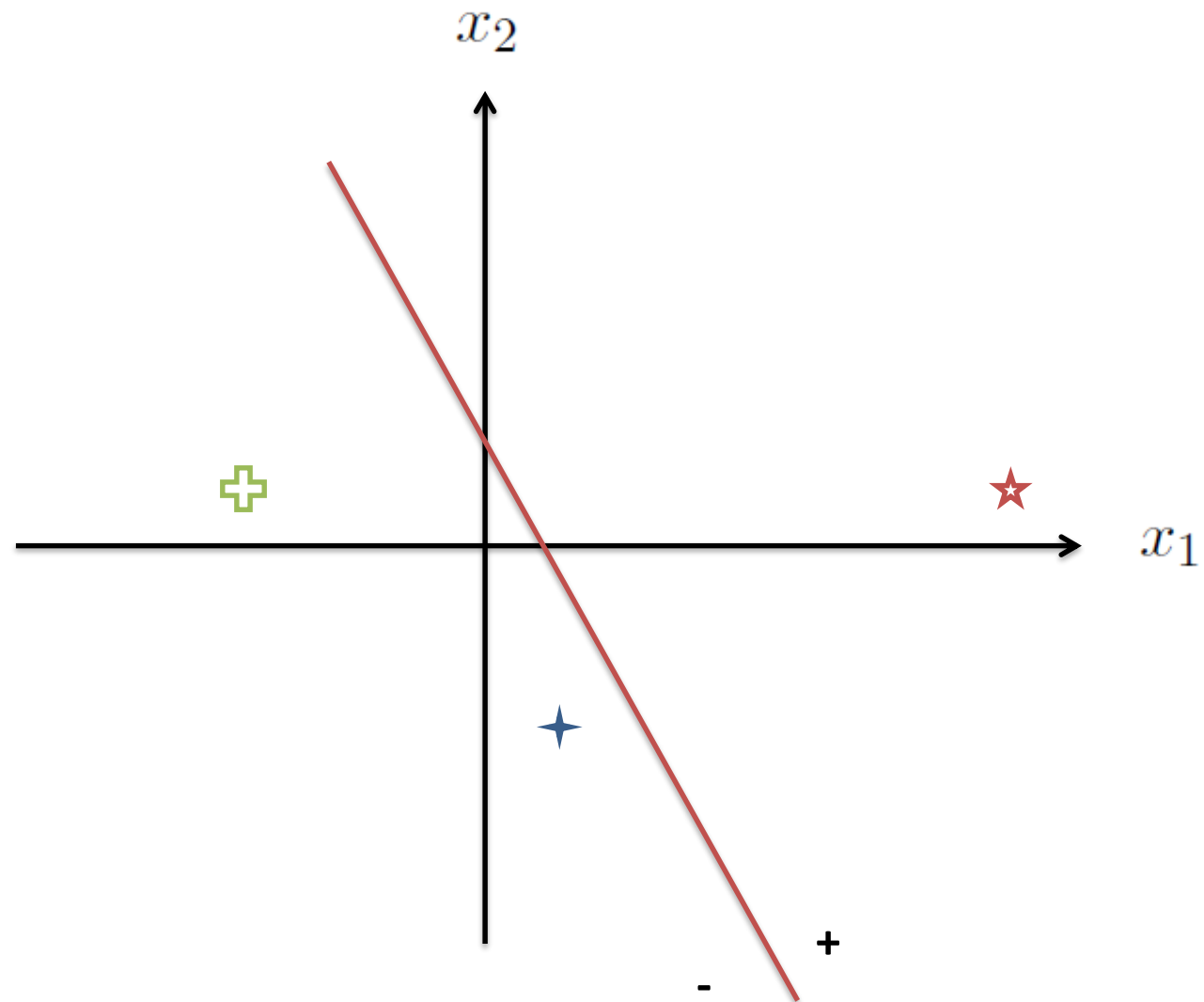
Last updated

End of Epoch 1

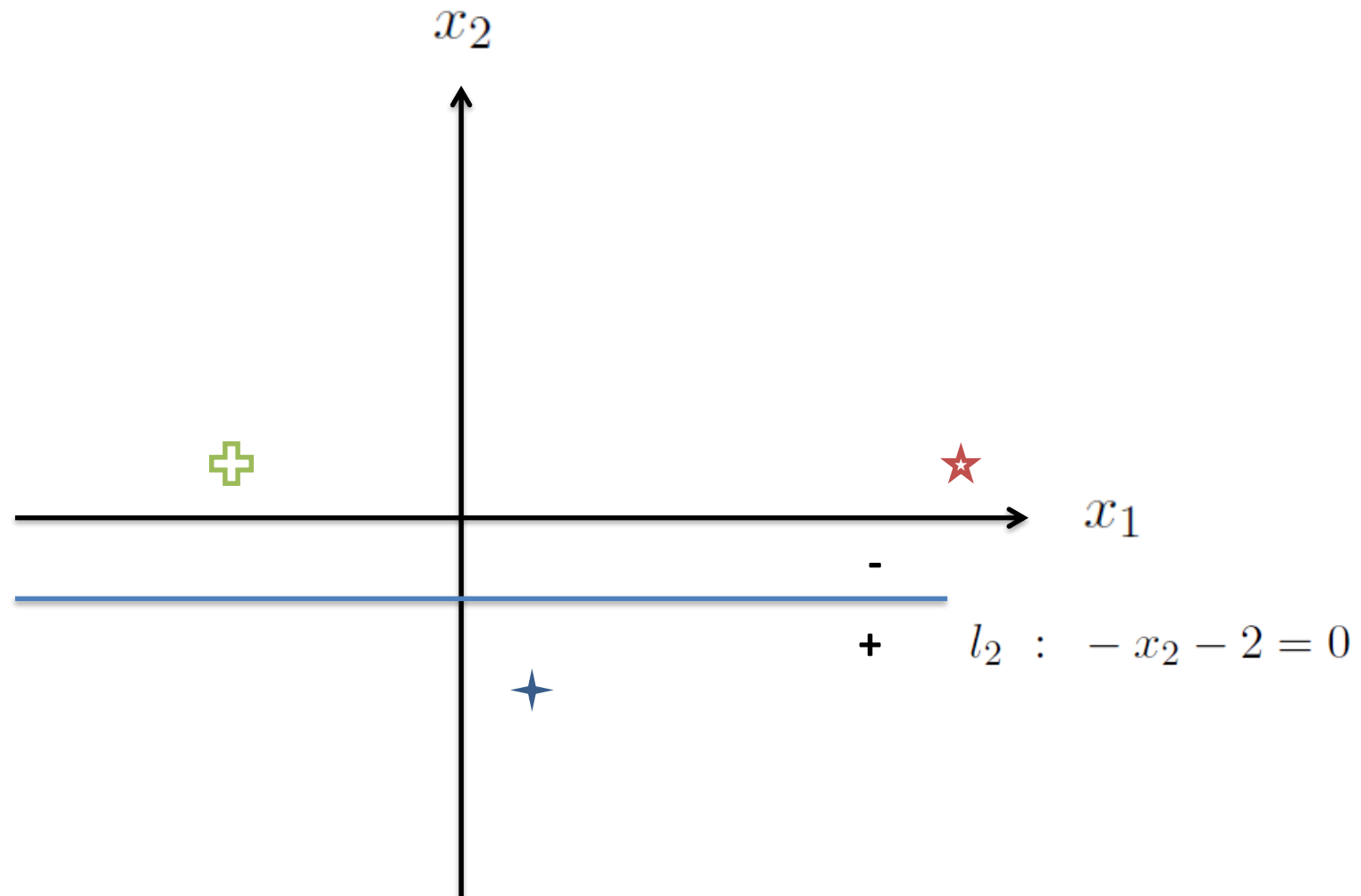
Last updated weights

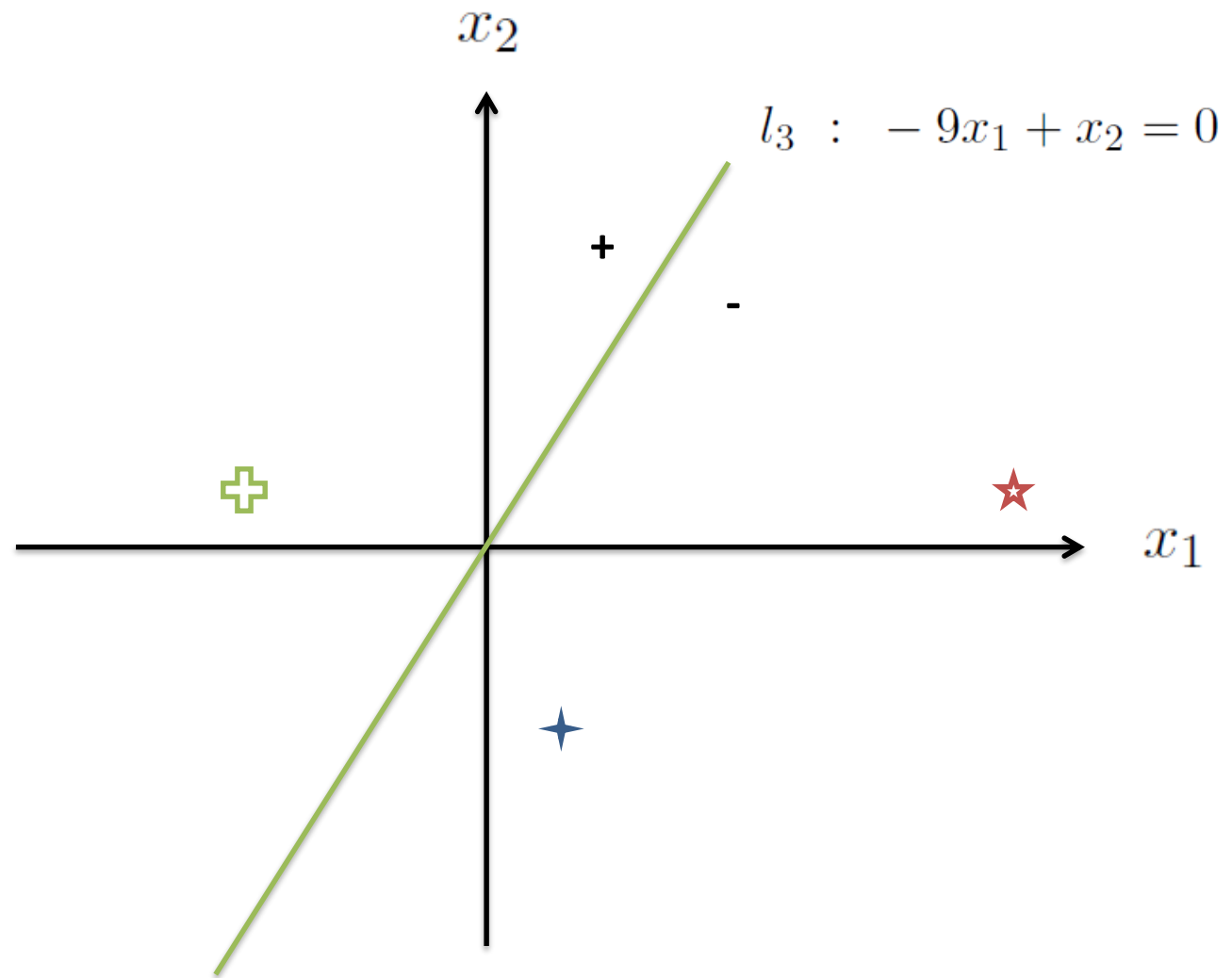
$$\bullet W_{1*} = \begin{pmatrix} 4 \\ 2 \\ 2 \end{pmatrix}, W_{2*} = \begin{pmatrix} 0 \\ -1 \\ 2 \end{pmatrix}, W_{3*} = \begin{pmatrix} -9 \\ 1 \\ 0 \end{pmatrix}$$

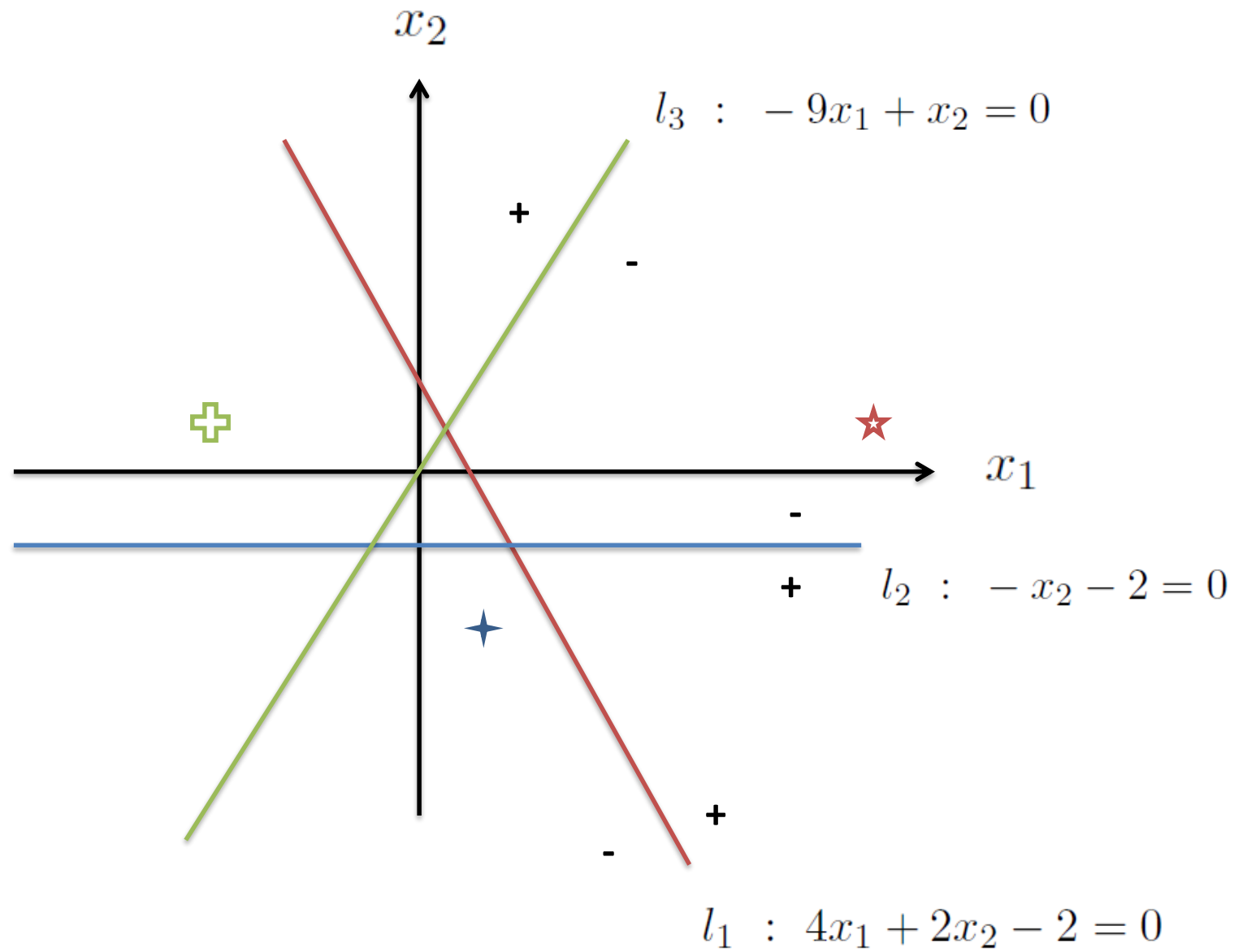




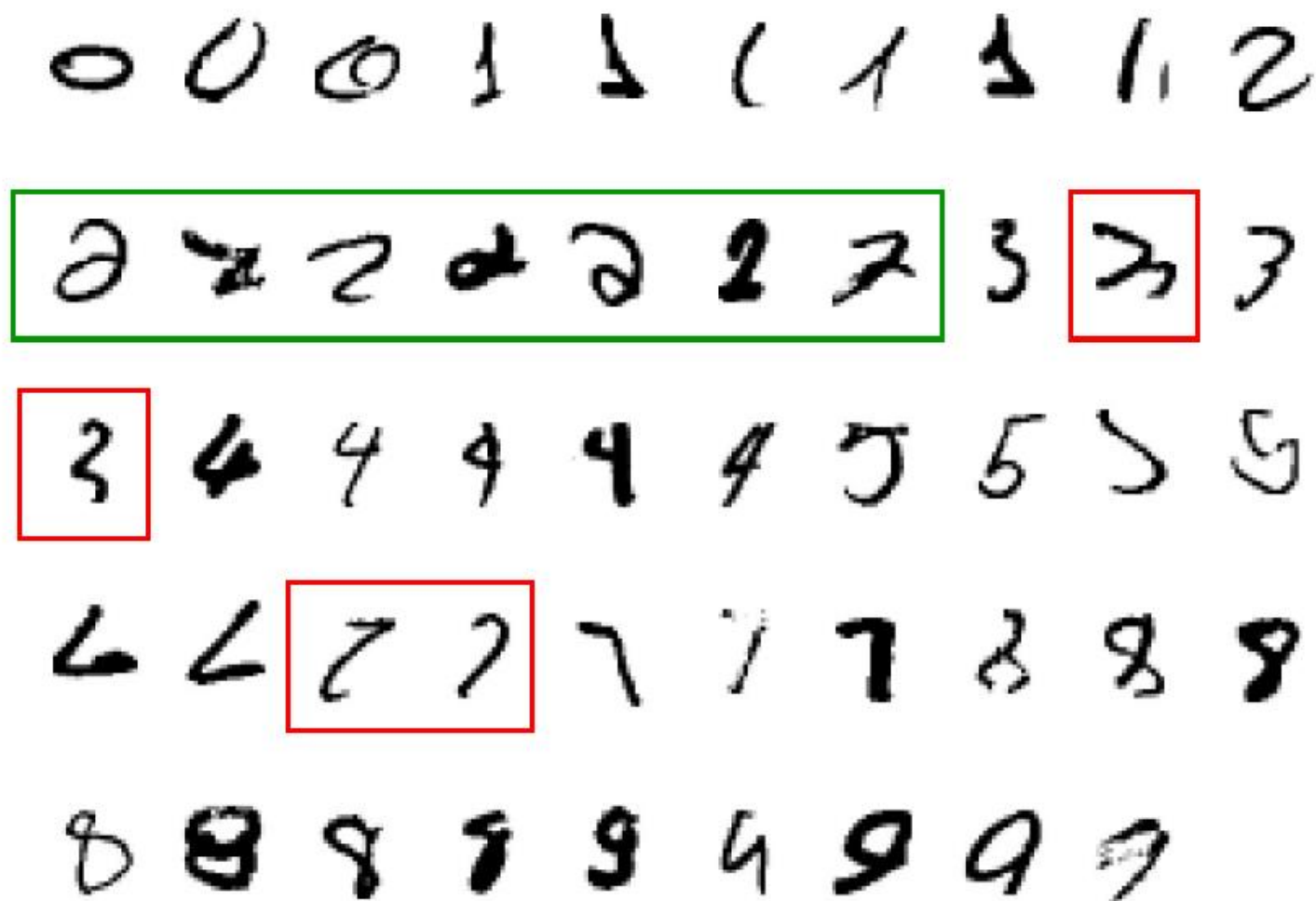
$$l_1 : 4x_1 + 2x_2 - 2 = 0$$





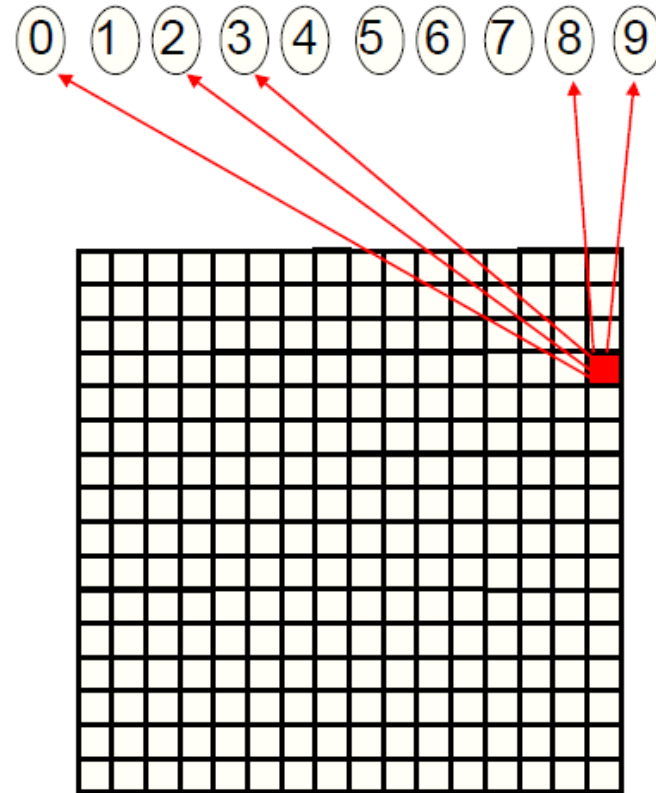


It is very hard to say what makes a 2

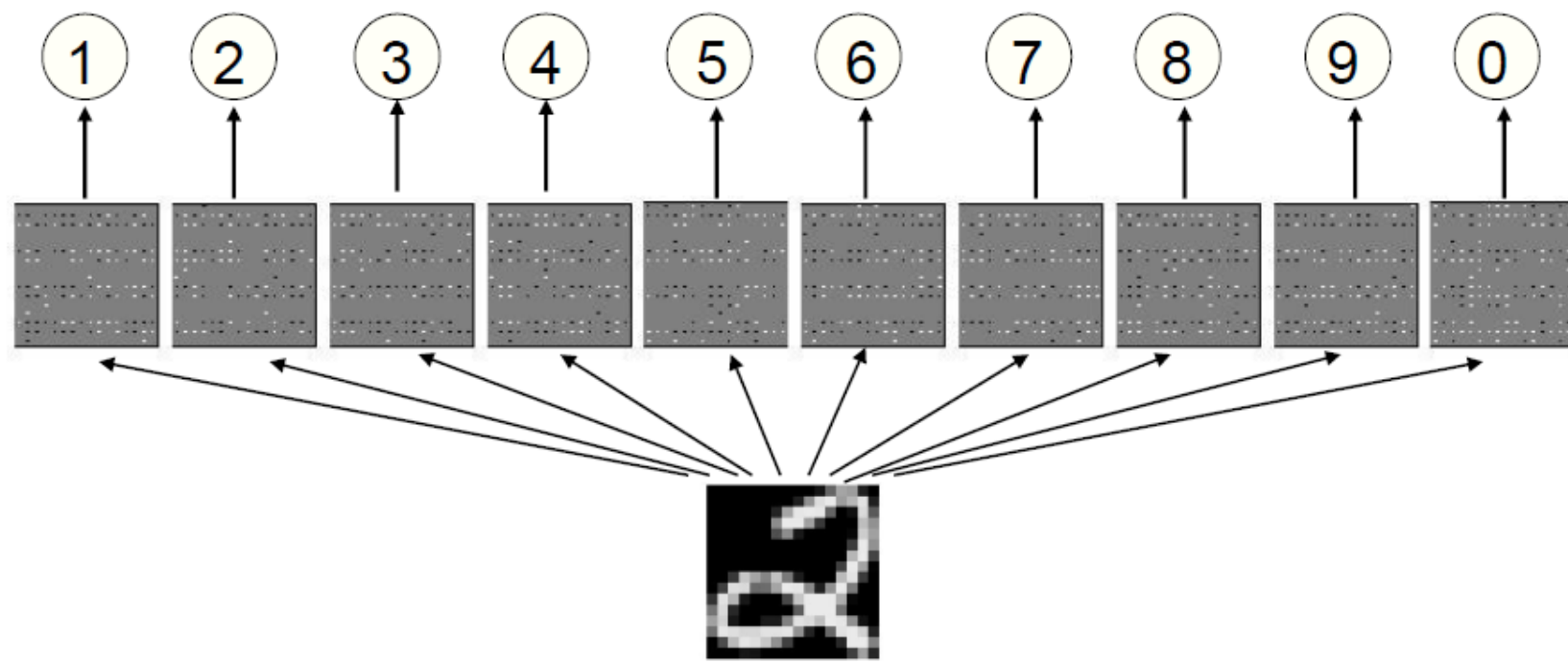


A very simple way to recognize handwritten shapes

- Consider a neural network with two layers of neurons.
 - neurons in the top layer represent known shapes.
 - neurons in the bottom layer represent pixel intensities.
- A pixel gets to vote if it has ink on it.
 - Each inked pixel can vote for several different shapes.
- The shape that gets the most votes wins.



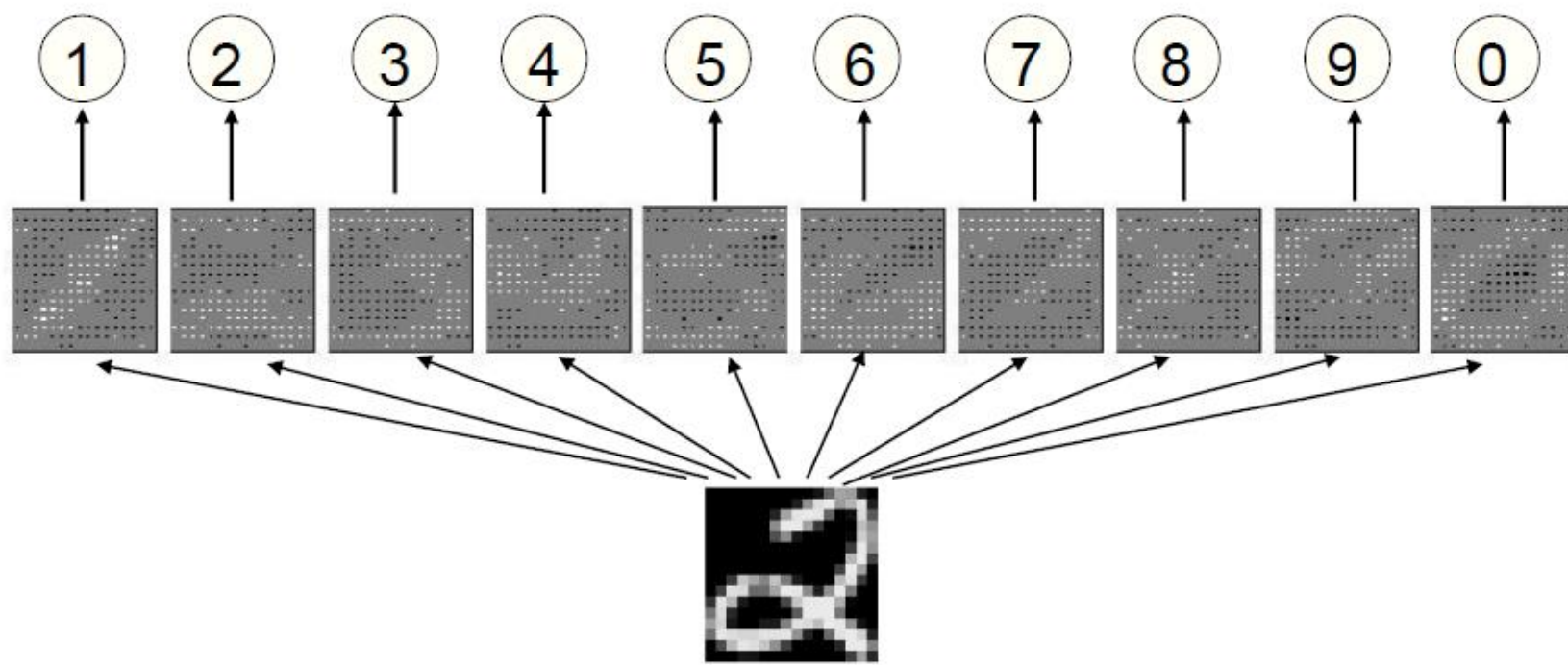
How to learn the weights



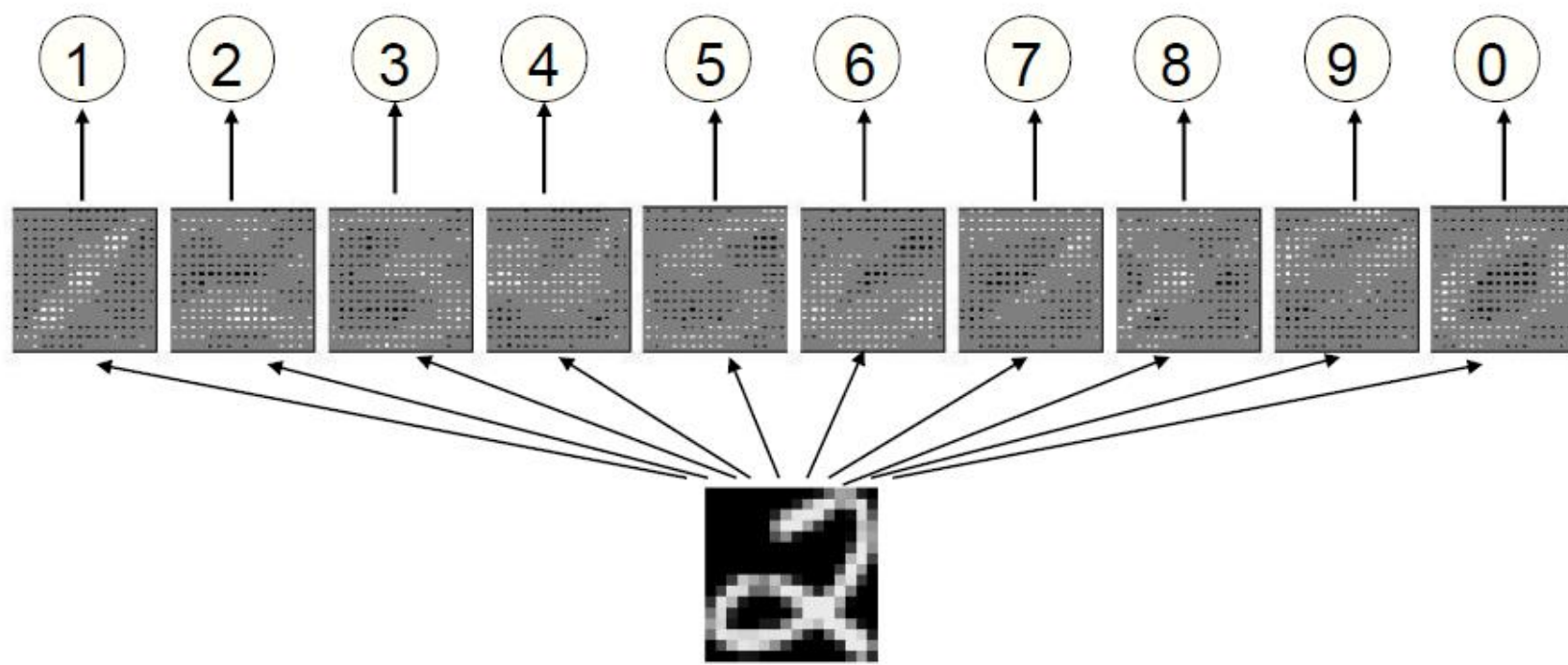
The image

Show the network an image and **increment** the weights from active pixels to the correct class.

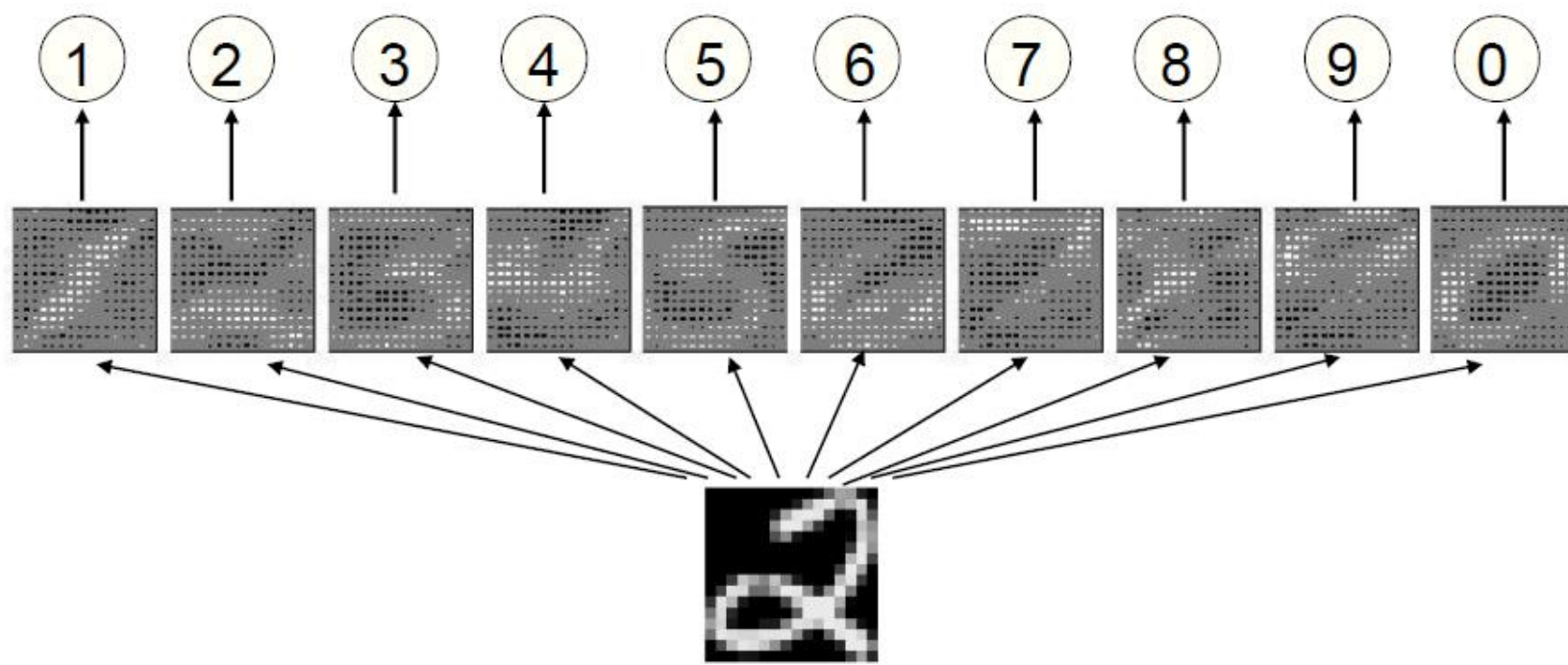
Then **decrement** the weights from active pixels to whatever class the network guesses.



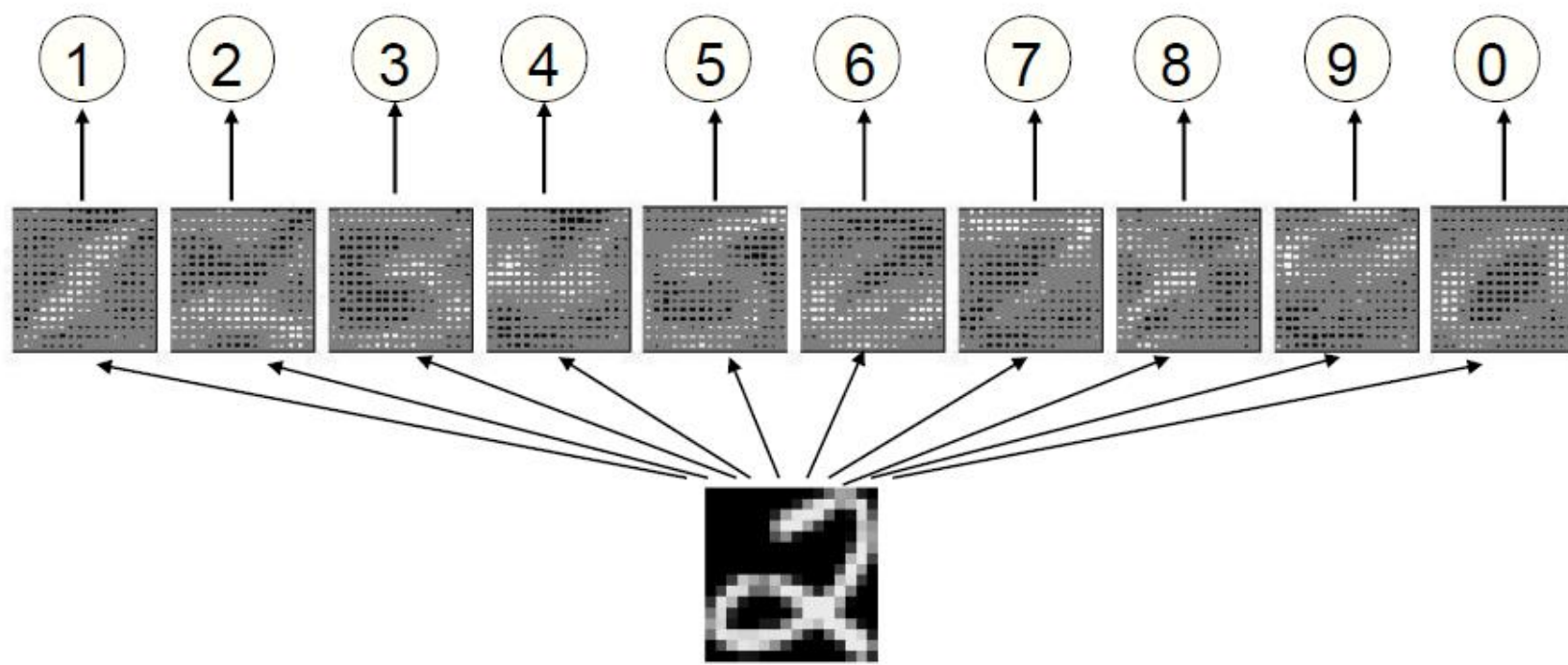
The image



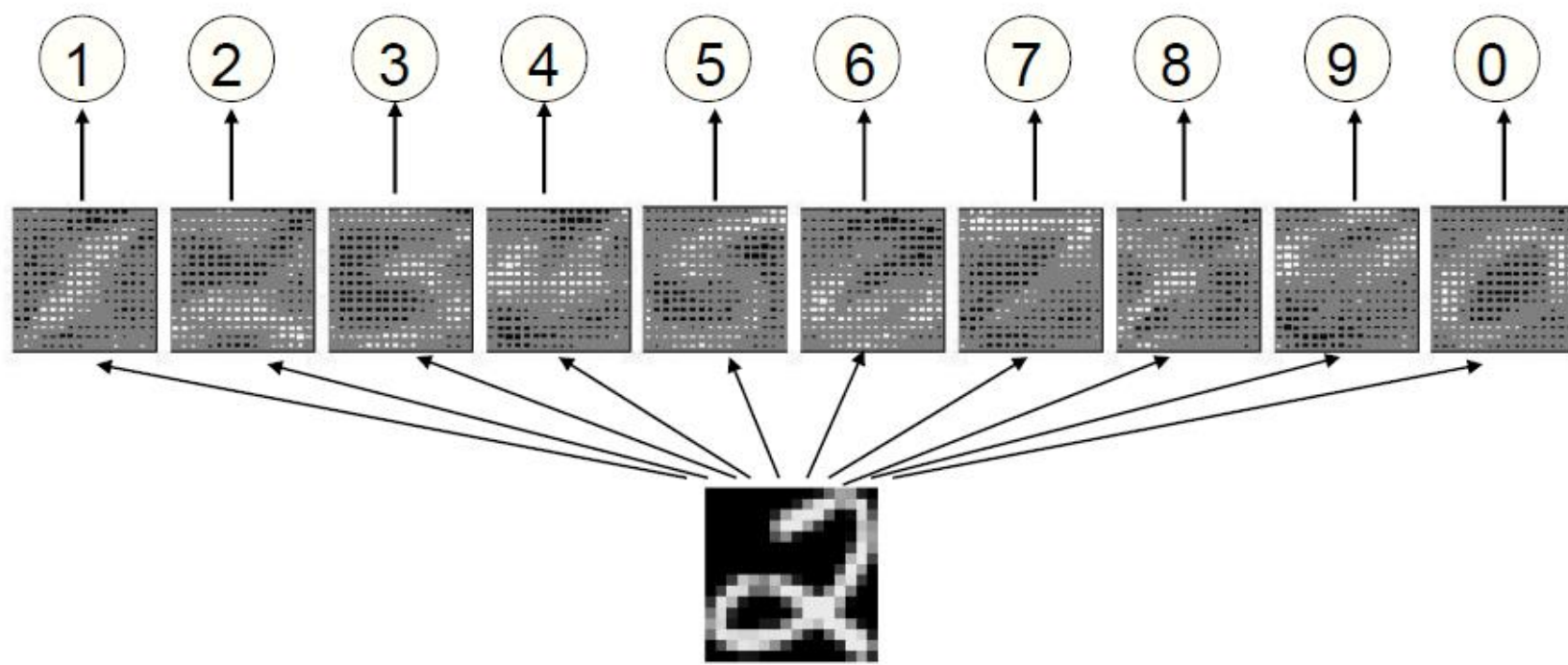
The image



The image



The image



The image

The learned weights

